

Mateus Molina

Ermöglichung von Hardware- und Software-in-the-Loop durch die Integration von BaSyx und MATLAB/Simulink

Projektarbeit

Institut für Messtechnik und Sensorik
Technische Universität Kaiserslautern

Betreuung

Dipl.-Ing. Paaranan Sivasothy
Prof. Dr.-Ing. Jörg Seewig

August 2022

DRAFT

Zusammenfassung

In der nachfolgenden Arbeit wird eine Anbindung des *BaSys* SDKs in die *MATLAB/Simulink* ausgelegt und entwickelt, mit dem Ziel, Anwendungsfälle wie u.a. die Realisierung von *Hardware-in-the-Loop* und *Condition Monitoring* abzudecken. Die erarbeitete Lösung ermöglicht die Erfassung und Aktualisierung von Daten aus *BaSys* AAS-Servers über eine vereinfachte Schnittstelle in Form von mit *C MEX S-function* entworfenen *Simulink*-Blöcken. Ergänzt wird diese durch Diagramme unterschiedlicher Betrachtungspunkte der Architektur und ein Beispielszenario, das exemplarisch zur Validierung der festgelegten Anforderungen eingesetzt wurde.

Schlagwörter: MATLAB/Simulink, BaSys, digitaler Zwilling, Hardware-in-the-Loop.

DRAFT

Inhaltsverzeichnis

1	Einleitung	1
1.1	Motivation	1
1.2	Zielstellung	2
1.3	Gliederung der Arbeit	2
2	Theoretische Grundlagen	3
2.1	Der Simulationsprozess	3
2.1.1	<i>Hardware-in-the-Loop (HiL)</i>	3
2.1.2	<i>Software-in-the-Loop</i>	4
2.2	MATLAB/Simulink	4
2.2.1	<i>Model-Based Systems Engineering (MBSE)</i>	4
2.2.2	Simulink	5
2.2.3	Funktionsweise von Simulink	5
2.2.4	<i>MATLAB S-functions</i>	6
2.2.5	Funktionsweise einer <i>C MEX S-function</i>	7
2.2.6	Struktur einer <i>S-function</i> in C++	8
2.2.7	Personalisierung der Blockbenutzeroberfläche	9
2.3	Industrie 4.0	10
2.3.1	<i>BaSys</i>	10
2.3.2	Eclipse BaSyx	12
2.4	REST-Schnittstellen	14
3	Planung und Durchführung	17
3.1	Entwicklungsprozess	17
3.1.1	Konzept und Anforderungsdefinition	17
3.1.2	Funktionale Spezifikation	18
3.1.3	Systemarchitektur	19
3.2	Entwurf	22
3.2.1	Entwicklungsumgebung	22
3.2.2	Wrappen des BaSyx als S-Funktion	22
3.2.3	A10 Ablesen des Wertes aus einem <i>SubmodelElement</i>	23
3.2.4	A20 Aktualisierung des Wertes aus einem <i>SubmodelElement</i>	23
3.2.5	A30 Abrufen von AAS-Daten aus einem Registry Server	24
3.2.6	A40 Anzeigen von Simulink-konformen Fehlermeldungen	25
3.2.7	A50 Anwendung von mehreren Blöcken in demselben Simulink-Modell	25
3.2.8	A60 Betriebsmodus wechseln	26
3.2.9	A70 Personalisierung der Blockbenutzeroberfläche	26
4	Validierung	33
4.1	Beispielszenario	33
4.1.1	Setup der <i>BaSyx</i> -Infrastruktur mit <i>Docker</i>	33
4.1.2	Aufbau	33

4.2 Ausführung und Prüfen der Anforderungen	36
5 Zusammenfassung und Ausblick	39
5.1 Fazit	39
5.2 Ausblick	39
Abbildungsverzeichnis	41
Tabellenverzeichnis	43

DRAFT

Kapitel 1

Einleitung

1.1 Motivation

Die Anforderungen der neuen Industrie 4.0 erfordern besondere Werkzeuge und Plattformen, die für eine wandelbare und flexible Produktion ausgedacht sind. *BaSys* und die Referenzimplementierung seiner Konzepte, *BaSyx*, legen Bausteine fest, auf die für diese Anwendung konforme Systeme ihrer Struktur basieren können. *BaSys 4.0* ist ein vom *BMBF* gefördertes Forschungsprojekt, das von *Fraunhofer IESE* gemeinsam mit 14 weiteren Partnern aus dem Bereich der Produktionstechnik entwickelt wird, um digitale Zwillinge als digitale Abbildungen für die Produktion zu realisieren **fraunhofer-institut furexperimentellesoftwareengineeringieseBaSysFlyer**.

Eine Open-Source Implementierung der von *BaSys* festgelegten Konzepte ist *Eclipse BaSyx*, eine von *BaSys* finanzierte *Middleware*, die wiederverwendbare Industrie 4.0 Komponenten und SDK in verschiedenen Programmiersprachen bereitstellt, wodurch die schnelle Entwicklung neuer Software-Komponenten ermöglicht wird **eclipsefoundationinc.BaSyxEclipsepedia**.

Das von *BaSyx* bereitgestellte SDK ist in verschiedenen Kontexten der Digitalisierung anwendbar, sei es für die Erstellung digitaler Zwillinge oder Vernetzung von mehreren unabhängigen Plattformen, die herkömmlich mit unterschiedlichen Datenformaten arbeiten. Eine Schnittstelle, die von Interesse für manche Ingenieure ist, ist die direkte Verbindung mit *MATLAB/Simulink*, wobei die mit diesen erstellten Modellen über eine Kommunikationsschnittstelle mit bestehenden *BaSyx*-Infrastrukturen verfügen können.

MATLAB/Simulink wird häufig in verschiedenen Industrie-Branchen von Ingenieuren großer und kleinerer Firmen eingesetzt, um Modelle mit wiederverwendbaren Komponenten und Bibliotheken zu simulieren **onlyftSimulinkMarketShare themathworksinc.SimulinkGettingStarted**. Simulink, zusammen mit BaSyx, könnte eingesetzt werden, um *Hardware-In-The-Loop*, bzw. *Software-In-The-Loop* leichter einzusetzen, sodass sich BaSyx um die Struktur der digitalen Zwillinge kümmern würde, während Simulink für die Modellierung des Verhaltens einzelner Bauteile oder für die Regelung gesamter Komponenten angewandt würde.

Nach Prüfung von verschiedenen Alternativen, um Simulink und BaSyx zusammen einzusetzen, hat sich herausgestellt, dass eine Kommunikationsschnittstelle zusätzliche Kenntnisse in C, REST und in der inneren Funktionsweise vom BaSyx voraussetzt, was ein limitierender Faktor für eine breitere Verwendung in der Industrie darstellen könnte. Eine unkomplizierte und erfolgreiche Verbindung der beiden Plattformen würde dem Stand der Technik folgende Vorteile bringen:

- Mehr Anreiz, früher im Projekt nach *BaSys* strukturierte digitalen Zwillingen einzusetzen
- vereinfachte und schnellere Anbindung von zu testenden Hardware- und Softwarekomponenten an ein MATLAB/Simulink-Modell;

- Reduktion der für die Erstellung von mit BaSyx verträglichen Modellen Lernkurve;
- wirksame Einbindung realer Daten in MATLAB/Simulink Modelle.

1.2 Zielstellung

Eine mögliche Lösung für die oben besprochene Problemstellung bietet eine *Simulink*-Bibliothek, die in der Lage ist, sich mit einem *BaSyx*-Server zu verbinden und das Abrufen und Aktualisieren von *AAS*-Eigenschaften in Echtzeit direkt in dieser Umgebung zu ermöglichen.

Dieses Projekt sieht die Entwicklung der oben beschriebenen Bibliothek vor. Anhand eines Anwendungsbeispiels, das im Wesentlichen aus einem Simulink-Modell besteht, wird der Nutzen dieser Bibliothek belegt. Unten wird der vorgesehene Umfang zusammengefasst:

1. Entwicklung einer Bibliothek für *MATLAB/Simulink*, um die Verbindung mit *BaSyx AAS* zu vereinfachen.
2. Einsatz der Bibliothek in einem Anwendungsbeispiel, das alle Anforderungen aus der Tabelle 3.1 erfüllt.

1.3 Gliederung der Arbeit

Im Kapitel 2 werden die für das Verständnis der Arbeit wesentlichen Grundlagen erklärt. Danach wird im Kapitel 3 sowohl eine detaillierte Beschreibung des Entwicklungsprozesses, als auch des Entwurfes behandelt. Der Schwerpunkt liegt dabei auf der Realisierung sämtlicher Anforderungen des Projekts. Im Kapitel 4 findet sich ein Beispielszenario, das das Testen wesentlicher Funktionalitäten des Systems ermöglicht hat. Letztendlich wird die Arbeit im Kapitel 5 zusammengefasst und ein Ausblick gegeben, wie mögliche Erweiterungen aussehen könnten.

Kapitel 2

Theoretische Grundlagen

In diesem Kapitel werden die für die Arbeit wesentlichen Konzepte näher beschrieben. Das Unterkapitel Simulation bietet einen Überblick über Begriffe wie *Software- und Hardware-in-the-Loop* bekommen. Anschließend werden *Simulink* und *Matlab* angesprochen, mit Schwerpunkt auf den technischen Aspekten zur Programmierung von *S-functions* mit *C MEX*. Das Thema Industrie 4.0 sowie auf diesen bezogenen Architekturen und Technologien wie *BaSys* und *BaSyx* werden in der Folge besprochen. Zum Schluss wird zusammenfassend die REST-Schnittstelle erläutert.

2.1 Der Simulationsprozess

Unter Simulation wird die Nachahmung der Funktionsweise eines realen Systems verstanden. Im Zentrum der Simulation steht ein Modell, Konstrukt, das die für die Anwendung wichtigsten Merkmale des ausgewählten Systems darstellt.

Sie finden Anwendung in unterschiedlichen Bereichen der Technik und nehmen eine wichtige Rolle bei der Lösung dynamischer und komplexer Probleme ein, die mittels analytischer Vorgehensweise nicht ohne weiteres gelöst werden können. Durch ihren Einsatz gewinnt der Nutzer Klarheit und Verständnis für den Entwicklungsprozess **zimmerDurchgaengigerSimulationsprozessZur2015**.

In den folgenden Unterkapiteln werden *Hardware-in-the-loop* und *Software-in-the-loop* vorgestellt, zwei Prozesse der Simulationstechnik, die im Produktentstehungsprozess häufig verwendet werden. Der Begriff *x-in-the-loop* deutet darauf hin, dass Abstraktionen eines Systems im Laufe des Entwicklungsprozesses eingesetzt werden, bzw. die Anbindung einer Hardware, sowie eine Software-Darstellung des Systems an die Simulationsumgebung.

2.1.1 *Hardware-in-the-Loop (HiL)*

Ein reales System durch Einsatz von physischen Prototypen ist oftmals wegen der Vielzahl an Konfigurationsmöglichkeiten nicht möglich. Eine Alternative dazu stellt die Anbindung der Hardware an die Simulationsumgebung durch eine elektrische Schnittstelle dar. Dieser Prozess wird als *Hardware-in-the-Loop (HiL)* bezeichnet. Im Wesentlichen wird ein reales Bauteil in einem virtuellen Versuch eingesetzt, und zwar schnell und ohne Risiken einzugehen, wie beispielsweise eine Beschädigung des Prototyps. **zimmerDurchgaengigerSimulationsprozessZur2015**

HiL ist seit Jahren in verschiedenen Branchen der Industrie, von Luft- und Raumfahrt bis zur Fahrzeugtechnik, eingesetzt, in denen Systeme besonders komplex sind. Der Einsatz von HiL bietet mehrere Vorteile, wie die Reduzierung der Kosten zum Testen und des mit Ausfall einhergehenden Risikos. Außerdem lassen sich damit, Komponenten früher im Produktlebenszyklus testen, da andere für das Testen notwendiger Komponente eventuell erst später in der Entwicklung verfügbar sein können. **ellisControlSystemDesign2012**

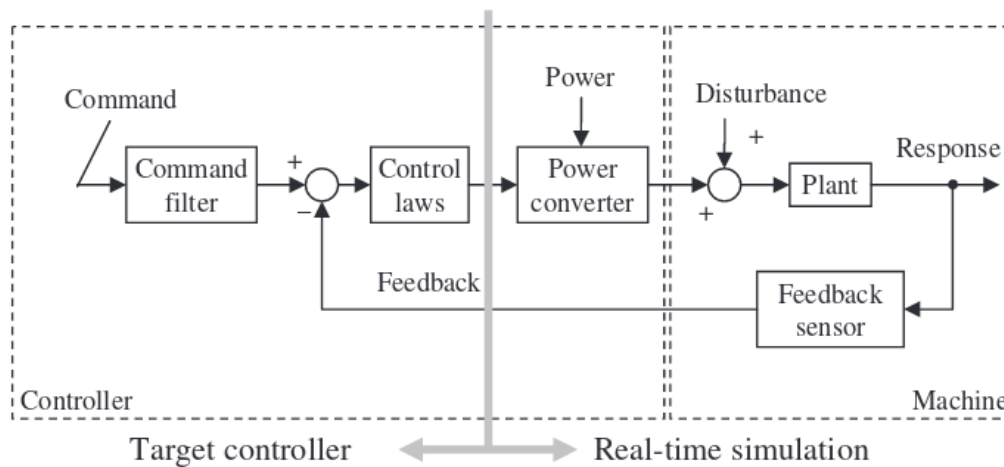


Abbildung 2.1: Blockschaltbild eines Regelungssystems, in dem der Regler im HIL getestet ist
ellisControlSystemDesign2012

Ein anderer wichtiger Vorteil dieses Ansatzes bezieht sich auf das Variantenmanagement. Varianten können mit Einsatz von *HiL* leichter erzeugt werden, unter Berücksichtigung, dass Anlagen oft variieren können, und die Prüfung von Mustern jeder Variante kann teuer sein. Mit Hilfe von *HiL* können daher Tests nur an einer Teilmenge der verfügbaren Varianten durchgeführt werden.
ellisControlSystemDesign2012

2.1.2 *Software-in-the-Loop*

Ein Nachteil des *Hardware-in-the-Loop* wäre die Verzögerung, bis die für das Projekt benötigte Hardware zur Verfügung steht **zimmerDurchgaengigerSimulationsprozessZur2015**. Eine Alternative für das Testen in früheren Phasen ist die Erstellung einer Software-Darstellung des Systems, die die für die jeweilige Phase notwendigen Anforderungen abdeckt. Damit lässt sich eine reine Software-Schnittstelle zwischen den Testkomponenten verwenden, im Gegensatz zu einer aufwändigeren elektrischen Schnittstelle, Schnittstelle im Falle des *HiL*-Ansatzes. **zimmerDurchgaengigerSimulationsprozessZur2015**.

2.2 MATLAB/Simulink

MATLAB ist ein leistungsfähiges Programm, um numerische Berechnungen in Ingenieurbereich durchzuführen. Diese Software findet Anwendung in der Berechnung, Modellierung und Simulation technischer Systeme und wird sowohl in unterschiedlichen Branchen der Industrie, als auch in der Universität eingesetzt. MATLAB bietet die Grundlage, um mathematische Berechnungen, Datenanalyse, Modellbildung und die Entwicklung von interaktiven Anwendungen zu realisieren **boslEinfuehrungMATLABSimulink2020**.

Dies setzt sich aus mehreren *toolboxes* zusammen, die jeweils bestimmte Bereiche der Ingenieurwissenschaften abdecken. Eine der bekanntesten *toolboxes* ist Simulink, die eine Blockdiagramm-Umgebung für Multi-Domäne-Simulationen und modellbasierten Entwurf bietet. Diese *toolbox* und der obere Begriff *Model-Based Systems Engineering* werden in dem nächsten Unterkapitel besser erläutert.

2.2.1 *Model-Based Systems Engineering (MBSE)*

Modellbasiertes Systems Engineering (engl.: *Model-Based Systems Engineering, MBSE*) ist die „formalisierte Anwendung von Modellbildung, um die Aktivitäten der Anforderungserfassung, Entwicklung, Verifikation und Validierung eines Systems, beginnend von der Konzeptphase, über die Entwicklungsphase bis zu späteren Lebenszyklusphasen zu unterstützen“ (**eignerModellbasierteVirtuelleProduktentwicklung**).

Modellen dienen zur vereinfachten Darstellung von komplexen Szenarien und können in Lauf des Entwicklungsprozesses eingesetzt werden. Dies steht im Gegensatz zu dem reinen Einsatz von textuellen Dokumenten, die oftmals zur Undeutlichkeit und Verkomplizierung eines Systems führen kann. Modelle finden häufig Anwendung bei der Anforderungserfassung, Entwicklung, Verifikation und Validierung eines technischen Projektes **eignerModellbasierteVirtuelleProduktentwicklung2014**.

2.2.2 Simulink

Simulink ist eine Blockdiagramm-Umgebung für die Auslegung von Systemen mit Mehrdomänenmodellen, Simulationen vor der Umstellung auf Hardware und Bereitstellung des Systems. Mit diesem Werkzeug werden der Entwurf auf Systemebene, die Simulation, die automatische Codegenerierung sowie den kontinuierlichen Test und die Verifikation eingebetteter Systeme unterstützt **themathworksinc.SimulinkGettingStarted**. Daher lassen sich *MBSE* und wichtige Aufgaben des Produktentstehungsprozesses wie Architekturwahl, detaillierter Entwurf, die Implementierung und das Testen durch dieses Werkzeuges realisieren **themathworksinc.SimulinkGettingStarted**.

Simulink bietet einen grafischen Editor, anpassbare Blockbibliotheken und *Solver* für die Modellierung und Simulation dynamischer Systeme. Dieses Werkzeug ist in MATLAB integriert, sodass das Einbinden von MATLAB-Algorithmen in Modelle und das Exportieren von Simulationsergebnissen zur weiteren Analyse nach MATLAB unterstützt werden **themathworksinc.SimulinkGettingStarted**.

Der fundamentale Baustein des Simulink ist der Block (Abb. 2.2), der aus einem Satz von Eingaben, Ausgaben, Zuständen und Parametern besteht.

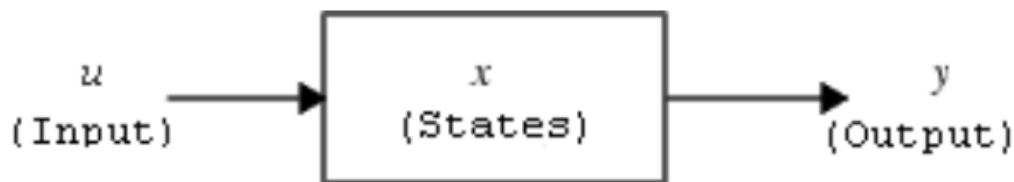


Abbildung 2.2: Struktur eines Simulink-Blockes **themathworksinc.WhatSFunctionMATLAB**

Blocks können miteinander verbunden und verschachtelt werden, wobei das Abbild eines Systems sofort sichtbar wird und das daraus resultierende Verhalten sich direkt in der Software validieren lässt.

2.2.3 Funktionsweise von Simulink

Wird ein Simulink Modell ausgeführt, laufen zwei Prozessen nacheinander durch: die Kompilierung und die Simulation. Bei dem ersteren wird die Logik der Blöcke in eine interne Darstellung umgesetzt, die mit dem *Simulink-Engine* interagieren kann. Andererseits werden in der Simulationsphase Zustände und Ausgabe berechnet. **SimulinkSimulationMATLAB**

Bei jedem Zeitschritt führt die Simulink-Engine folgende Schritte durch:

1. **Berechnung der Modellausgaben:** Die Modell-Outputs-Methode wird aufgerufen. Die Modell-Outputs-Methode ruft wiederum die System-Outputs-Methode des Modells auf, die die Outputs-Methoden der Blöcke aufruft. Diese übergibt jeder Block-Outputs-Methode die folgenden Argumente: einen Zeiger auf die Datenstruktur des Blocks und auf seine *SimBlock*-Struktur. Die *SimBlock*-Datenstrukturen verweisen auf Informationen, die die Outputs-Methode benötigt, um die Ausgaben des Blocks zu berechnen, einschließlich der Position seiner Eingabepuffer und seiner Ausgabepuffer.
2. **Berechnung der Zustände des Modells:** Die Simulink-Engine berechnet die Zustände eines Modells, indem sie einen *Solver* aufruft. Welcher *Solver* aufgerufen wird, hängt davon

ab, ob das Modell keine, diskrete und/oder kontinuierliche Zustände hat. Dadurch können zwei Anwendungsfälle unterschieden werden:

- Wenn das Modell nur diskrete Zustände hat, ruft die Simulink-Engine den vom Benutzer ausgewählten diskreten *Solver* auf. Dieser berechnet die Größe des Zeitschritts, der erforderlich ist, um die Abtastzeiten des Modells zu erreichen. Anschließend ruft er die Update-Methode des Modells auf. Diese ruft hingegen die Update-Methode seines Systems auf, die die Update-Methoden der einzelnen Blöcke, die das System enthält, in der Reihenfolge aufruft, die durch die in der Link-Phase erstellten Update-Methodenlisten festgelegt ist.
 - Wenn das Modell nur kontinuierliche Zustände hat, ruft die Simulink-Engine den vom Modell angegebenen kontinuierlichen *Solver* auf. Je nach *Solver* wird entweder einmalig die Methode Derivatives des Modells oder tritt in einen Unterzyklus *Minor*-Zeitschritte aufgerufen, in dem dieser wiederholt die Outputs-Methoden und die Derivatives-Methoden des Modells aufruft, um die Outputs und Derivatives des Modells in aufeinander folgenden Intervallen innerhalb des *Major*-Zeitschritts zu berechnen. Dies geschieht, um die Genauigkeit der Zustandsberechnung zu erhöhen.
3. **Optional wird auf Unstetigkeiten in den kontinuierlichen Zuständen der Blöcke geprüft.**
4. **Berechnung der Zeit für den nächsten Zeitschritt** im Laufe jedes Simulationsschrittes berechnet *Simulink* einen neuen Zeitschritt durch Aufsummierung mit einem dynamischen Zeitdifferenzial basiert auf den eingeschätzten Fehler zwischen der simulierten Lösung und der aktuellen. Am Ende der Simulation ist ein Vektor mit aus dem Vorgang resultierenden Zeit, Zustand und Ausgabe ausgegeben. **SimulationPhasesDynamic**

Die Schritte 1 bis 4 werden so lange wiederholt, bis die Stoppzeit der Simulation erreicht ist. Im Abbild 2.3 wird diese allgemeine Struktur grafisch dargestellt.

2.2.4 MATLAB S-functions

S-Functions (System-Funktionen) sind ein leistungsfähiger Mechanismus zur Erweiterung der Fähigkeiten der Simulink-Umgebung. Eine *S-function* ist eine in einer der unterstützten Programmiersprachen von *MATLAB* (*MATLAB*, C, C++ oder Fortran) geschriebene Beschreibung eines Simulink-Blocks in Computersprache **themathworksinc.WhatSFunctionMATLAB**.

Sie werden in jedem Simulationsschritt ausgeführt und können in unterschiedlichen Momenten angewandt werden, z.B. an der Initialisierung, Aktualisierung und Terminierung, sowie bei Eingeben- und Ausgeben-Event eines Blockes. Damit lässt sich personalisiertes Verhalten in Simulink einbetten, wie z.B. die Verbindung mit anderen Plattformen, wodurch der Austausch vom Parametersatz ermöglicht wird.

Für den Entwurf von *S-functions* werden Benutzern zwei *API* zur Verfügung gestellt: *Level-1 und Level-2 MATLAB APIs* und *C MEX S-functions*, wobei das Erstere zwei unterschiedliche Varianten anbietet, eine (Level-1) mit eingeschränkten Funktionalitäten und für einfache Anwendung geeignet, und andere (Level-2) für ein umfangreicheres Set. Die *C MEX API* entspricht großteils dem Umfang der Level-2 API, jedoch mit dem Hauptunterschied, dass sie für den Entwurf *S-functions* mit C/C++ als Programmiersprache geeignet ist. Mithilfe dieser APIs gibt es mehrere Möglichkeiten eine *S-Function* auszulegen, wie zum Beispiel:

- direkt im *MATLAB*-Editor, durch die Anwendung der *MATLAB*-Programmiersprache
- der Einsatz von einer der unterstützten Programmiersprachen (z.B. C/C++) über *MEX API*
- die Benutzung von *S-Function builder*, welches eine grafische Benutzeroberfläche für die Programmierung *S-functions* bietet.

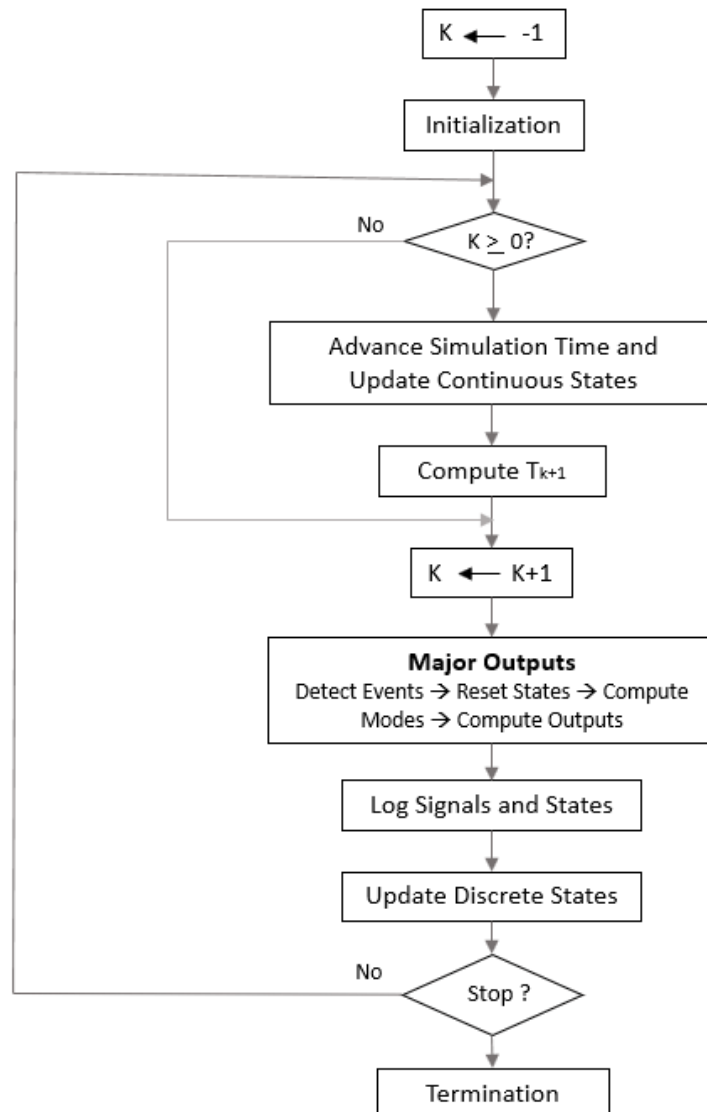


Abbildung 2.3: Struktur eines Simulationssdurchlaufes in Simulink, wobei k als Zähler für den *Major*-Zeitschritt dient

Mit der zweiten Methode erstellten *S-functions* werden für jeden Block eine *MEX*-Datei erzeugt, die sich wie ein normales *MATLAB*-Script verhalten, allerdings eine Anbindung mit *C/C++*-Methoden aufweisen. Diese sind grundsätzlich dynamisch gelinkte Subroutinen, welche innerhalb von *MATLAB* geladen und ausgeführt werden können. Ein dynamisches gelinktes Programm sucht nach den *runtime libraries*, sobald die Datei ausgeführt wird.

2.2.5 Funktionsweise einer *C MEX S-function*

Eine mit *C MEX* erstellte *S-function* ist von der *Simulink-Engine* als eine Reihenfolge von *Callback*-Methoden verstanden. Die Logik für die Initialisierung, Terminierung, als auch die Handhabung beider diskreten und kontinuierlichen Arbeitsschritte ist innerhalb dieser beschrieben. Zwei wichtige Prozesse finden bei jeder Simulation unter Anwendung von *C MEX S-functions* statt, nämlich die Initialisierung des Modells und die Simulationsschleife. **SimulinkEngineInteraction**

In Abb. 2.4, bzw. 2.5 wird ein Überblick der *Callback*-Methoden in der Reihenfolge gezeigt, mit der diese bei der Engine aufgerufen werden. Ausgefüllte Rechtecke kennzeichnen *callbacks*, die immer während der Modellinitialisierung oder bei jedem Zeitschritt auftreten. Gestrichelte Rechtecke zeigen *callbacks* an, die während der Initialisierung und/oder bei einigen oder allen Zeitschritten während der Simulationsschleife auftreten können.

Bei der Initialisierung konfiguriert die Engine die *S-function* für eine bevorstehende Simulation. Die Engine führt immer die erforderlichen Aufrufe an *mdlInitializeSizes* und *mdlInitializeSampleTime* aus, um die grundlegenden Attribute der *S-function* einzurichten, einschließlich Eingangs- und Ausgangsports, Dialogparameter der *S-function*, Arbeitsvektoren, Abtastzeiten usw.

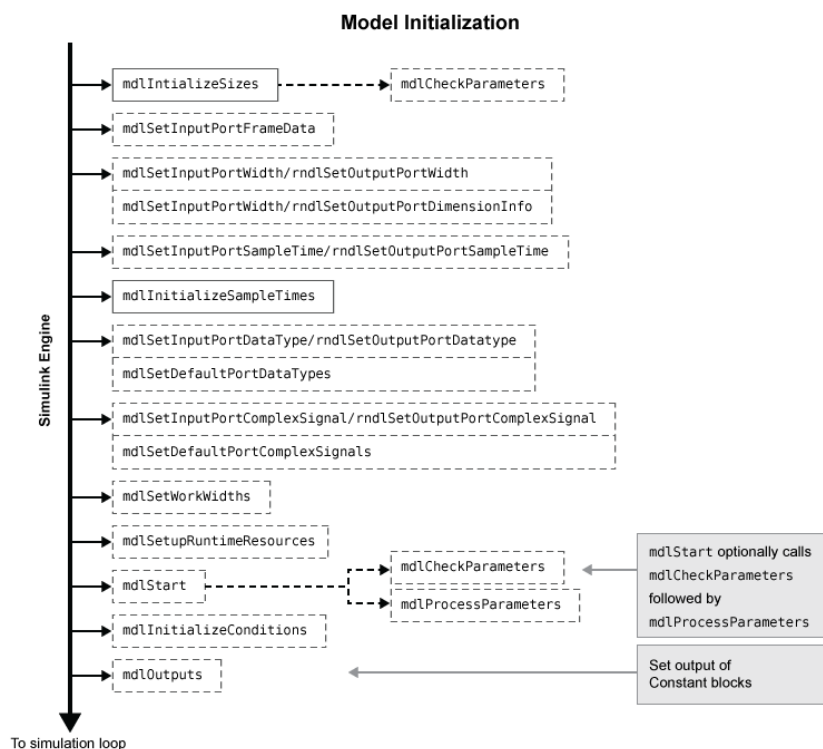


Abbildung 2.4: Ablaufreihenfolge, wenn das Modell initialisiert ist **SimulinkEngineInteractiona**

Nach diesem Schritt führt die Simulink-Engine die Simulationsschleife aus. Wenn diese unterbrochen wird, entweder manuell oder durch einen Fehler, springt die Engine direkt zur Methode *mdlTerminate*. Wurde die Simulation manuell angehalten, schließt die Engine zunächst den aktuellen Zeitschritt ab, bevor sie *mdlTerminate* aufruft.

2.2.6 Struktur einer *S-function* in C++

Im Quellcode 1 wird ein Beispiel einer *C MEX S-function* gezeigt. Der Veranschaulichung halber gibt diese den gedoppelten Wert der Eingabe aus. Am Anfang des Codes lassen sich die *define*-Anweisungen erkennen, wobei dort der Name der Funktion und danach das *API-Level* (Level-1, bzw. Level-2) festgelegt werden. Auch wichtig ist die Anweisung `#include "simstruc.h"`, die zur Importierung der *header*-Datei dient, welche Zugriff auf die *SimStruct*-Datenstruktur, sowohl die *MATLAB-API* gibt.

Darüber hinaus lässt sich im Beispiel 4 *MEX*-Methoden finden, die immer definiert werden müssen:

- *mdlInitializeSizes*: legt die Anzahl der Eingangs- und Ausgangsports, die Größe der Ports und andere Informationen (wie die Anzahl der Zustände) fest

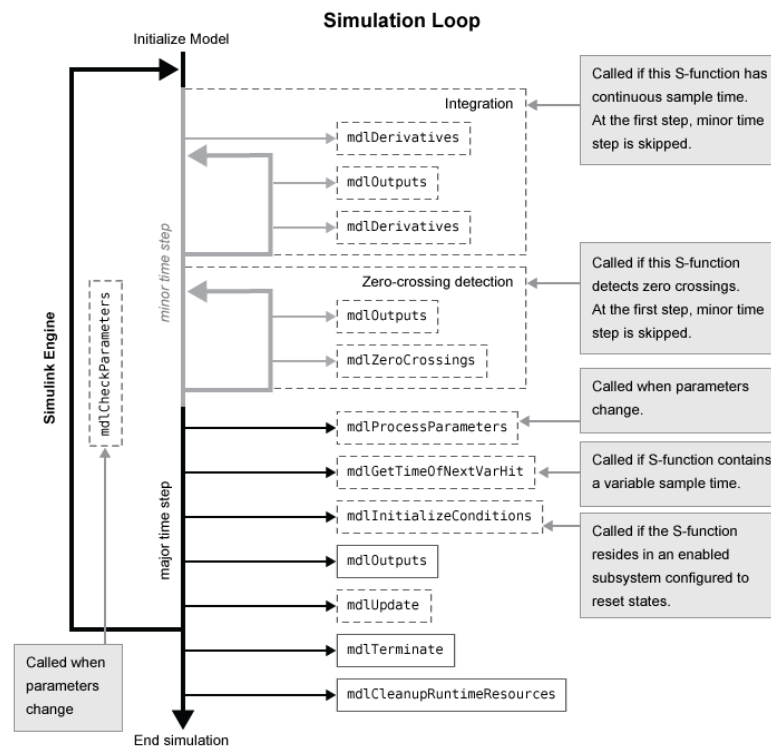


Abbildung 2.5: Simulationsschleife einer mit C MEX erstellten S-function **SimulinkEngineInteraction**

- *mdlInitializeSampleTimes*: legt die Abtastzeiten der S-Funktion fest
- *mdlOutputs*: wird von der Engine bei jedem Zeitschritt aufgerufen, um die Blockausgaben zu berechnen. Im Beispiel multipliziert diese das Eingangssignal mit 2 und schreibt die Antwort in den Ausgang.
- *mdlTerminate*: wird am Ende der Simulation ausgeführt und könnte beispielsweise zur Speicherfreigabe dienen. **CreateBasicMEX**

2.2.7 Personalisierung der Blockbenutzeroberfläche

Simulink bietet die Möglichkeit an, die Benutzeroberfläche, auch im Kontext als *Mask* bezeichnet, der mit *S-function* erstellten Blöcken zu personalisieren und dadurch die Eingabe von den von ihnen benötigten Parametern einfacher zu gestalten. **MaskingFundamentalsMATLAB** außerdem ist damit auch möglich:

- das Symbol des Blockes zu personalisieren
- die Bereitstellung eines Dialogfelds, das nur den Zugriff auf ausgewählte Parameter der zugrunde liegenden Blöcke ermöglicht
- die Bereitstellung einer benutzerdefinierten Beschreibung, die speziell auf den maskierten Block zugeschnitten ist
- die Initialisierung von Parametern mit *MATLAB*-Code

In der Abb. 2.6 ist die Bedieneroberfläche zur Einstellung der *Mask* gezeigt. Auf der linken Seite ist der Auswahl und Einfügen von mehreren Komponenten möglich, während ihre Parameter in den Fenstern in der Mitte und auf der rechten Seite konfiguriert werden können. Dies erlaubt dazu

die Einstellung von internen *callback*-Funktionen, in denen Validierungsregeln eingebettet werden können, wie z.B., nachdem der Benutzer mit einer Komponente interagiert hat, wird automatisch ein anderes Dialogfeld ausgeblendet.

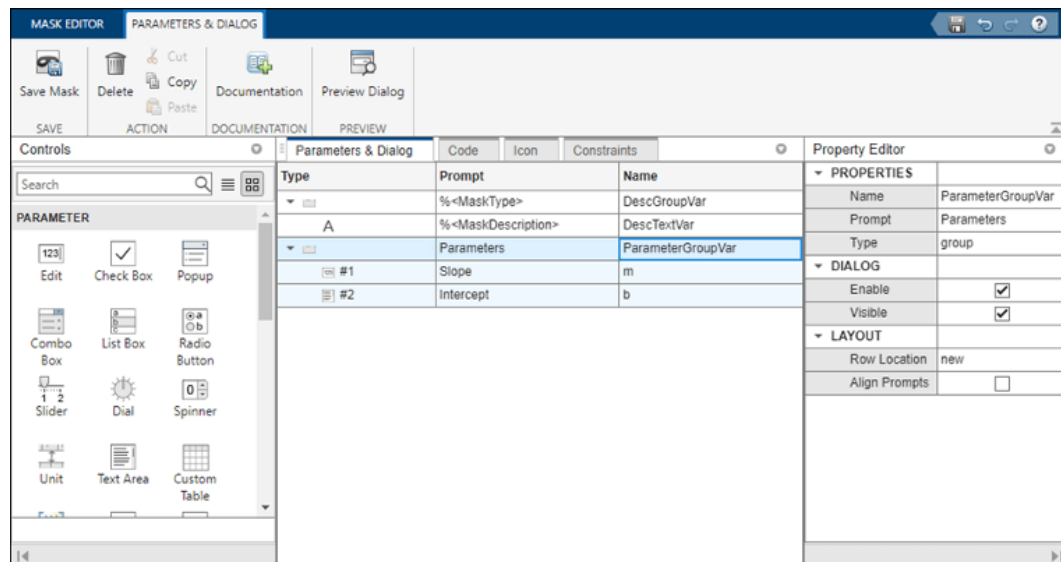


Abbildung 2.6: Dialog zur Einstellung der Blockbenutzeroberfläche MaskingFundamentalsMATLAB

2.3 Industrie 4.0

Industrie 4.0 (I4.0) ist die bevorstehende vierte große Veränderung in der Fertigung. Sie wurde durch die fortschreitende Digitalisierung ausgelöst und befasst sich mit den Erfordernissen einer wandelbaren Produktion in einer sich immer schneller verändernden Welt **kuhnServicebasedArchitecturesProduction2020**.

Industrie 4.0 ermöglicht sowohl eine effizientere als auch eine flexiblere Produktion. Die Automatisierung aktueller Produktionssysteme wird häufig mit speicherprogrammierbaren Steuerungen (SPS) realisiert. Diese automatisieren sich wiederholende Aufgaben, die in zyklischen Programmen ausgeführt werden. Sie führen eine Reihe von Fertigungsschritten für jedes Werkstück aus, z. B. eine Abfolge von Fräs- oder Bohrvorgängen. Später kamen weitere Systeme hinzu, z. B. Roboter mit eigenen Betriebssystemen, aber auch ERP-Systeme (*Enterprise Resource Planning*) und MES (*Manufacturing Execution Systems*), um das Ressourcenmanagement zu verbessern und eine flexiblere Produktion von Produktvarianten zu ermöglichen **kuhnServicebasedArchitecturesProduction2020**.

Die vierte industrielle Revolution ist die durchgängige Digitalisierung der Produktion. Dabei geht es nicht um die Substitution von SPS oder die Einführung neuer Geräte, sondern um neue Systemarchitekturen, die eine *Peer-to-Peer*-Kommunikation ermöglichen, um beispielsweise eine vorausschauende Wartung, eine KI-basierte Anomalie-Analyse in der Produktion und eine effiziente Produktion kleiner Losgrößen zu unterstützen. Bei Industrie 4.0 geht es also nicht um die Integration einer einzigen neuen Technologie. Vielmehr geht es um die Einführung neuer Architekturparadigmen, die eine flexiblere und wandelbarere Produktion ermöglichen **kuhnServicebasedArchitecturesProduction2020**.

2.3.1 BaSys

BaSys 4.0 ist ein Plattform-unabhängiges modulares System basiert auf die Struktur der Referenzarchitektur *RAMI4.0*. Damit wird die Realisierung eines Grundlagersystems für die Produktionsanlagen angestrebt und die folgenden Vorteile abgezielt:

- Bereitstellung und Umsetzung der primären Industrie 4.0-Konzepte;

- einfache dynamische Produktionsänderungen;
- einfache Realisierung digitaler Zwillinge über definierte Schnittstellen;
- vorausschauende Wartung - Online-Zugriff auf die Prozessdaten;
- einfache Integration bestehender und neuer Geräte **eclipsefoundationinc.BaSysxEclipsepedia**

Die Realisierung von *BaSys 4.0* basiert auf drei Haupttechnologien, die als zentrale Säulen der Industrie 4.0-konformen Produktionsarchitektur bezeichnet werden: der virtuelle Automatisierungsbus (VAB), die Verwaltungsschale (AAS), und die Steuerungskomponenten (SPS) **eclipsefoundationinc.BaSysxEclipsepedia**

In der Abb. 2.7 werden die Hauptkomponenten eines *BaSys 4.0*-Projektes dargestellt.

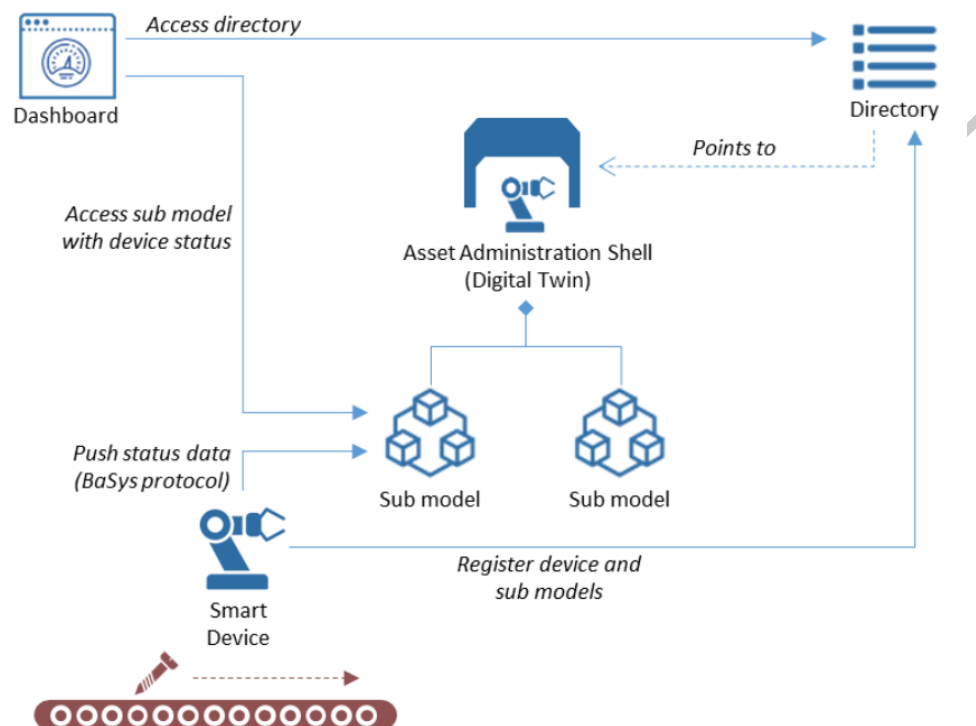


Abbildung 2.7: Beispiel einer BaSys-Architektur **eclipsefoundationinc.BaSysxEclipsepedia**

Verwaltungsschale (AAS) / digitaler Zwilling Die Verwaltungsschale (in Englisch *Asset Administration Shell*, oder *AAS*) oder digitaler Zwilling stellt das digitale Abbild eines physischen Gegenstands dar und stellt die Schnittstelle für die Kommunikation in der digitalen Umgebung zur Verfügung. **plattformindustrie4.0DiskussionspapierVerwaltungsschalePraxis**

Jedes relevante Gut (auch als *Asset* bezeichnet), z. B. Geräte, Arbeitskräfte, Produkte, aber auch der Herstellungsprozess, enthält genau eine *AAS*. Die *AAS* wird durch einen eindeutigen Bezeichner identifiziert, der genau dieses *Asset* repräsentiert. Aufgrund dieser Vielfalt an dargestellten Objekten enthält jede *AAS* einen unterschiedlichen Satz von *submodels*. *Submodels* sind Datenmodelle, die bestimmte Aspekte einer Anlage beschreiben, z. B. ihr digitales Typenschild, die angebotenen Dienstleistungen oder die Anlagentopologie. Eine *AAS* und ihre *submodels* sind letztendlich eine einheitliche Darstellung einer realen Anlage. **eclipsefoundationinc.BaSysxEclipsepedia**

In der Abb. 2.8 werden den Hauptelementen eines AAS dargestellt, bzw. das *Asset*, die *submodels* und ihre Eigenschaften (*properties*), und das Ganze in derselben einheitlichen Umgebung einer *AAS* repräsentiert.

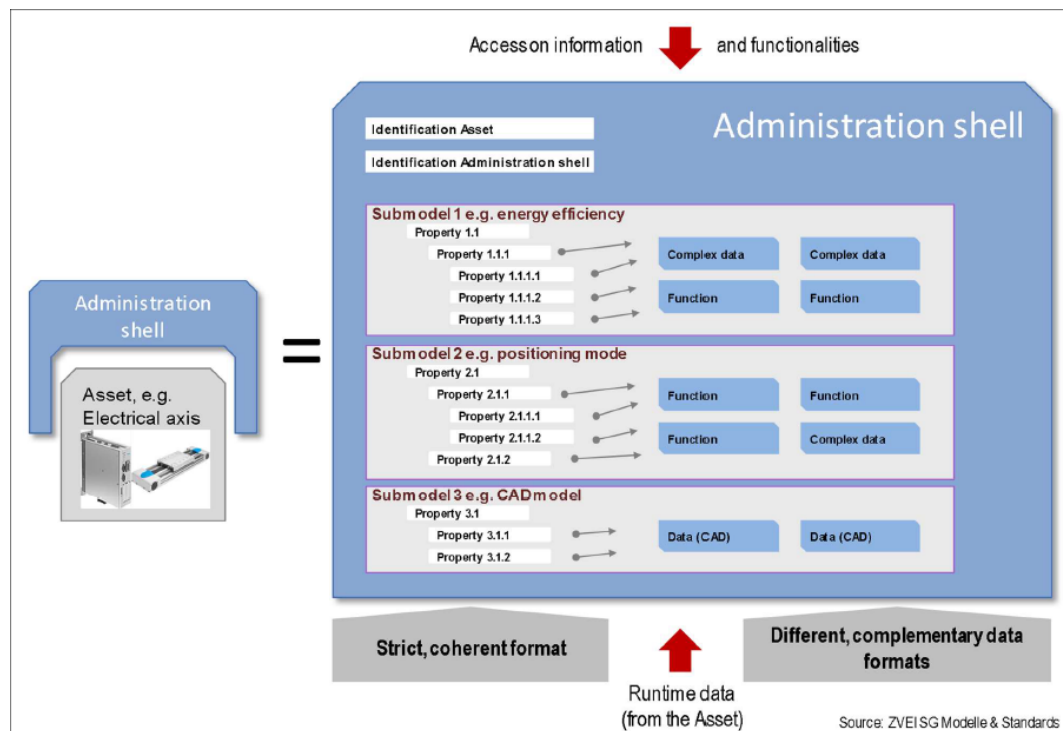


Abbildung 2.8: Abbild einer AAS eclipsefoundationinc.BaSyxEclipsepedia

Automatisierungsbuss (VAB) Der *BaSys 4.0 virtual automation bus (VAB)* ermöglicht die direkte Kommunikation über die verschiedenen Schichten der Automatisierungsstruktur. Geräte in der Fertigung können mit ERP-Systemen (*enterprise resource planning*) interagieren, selbst wenn die anderen Kommunikationsprotokollen anwenden fraunhofer-institut furexperimentellesoftwareengineeringieseBaSy

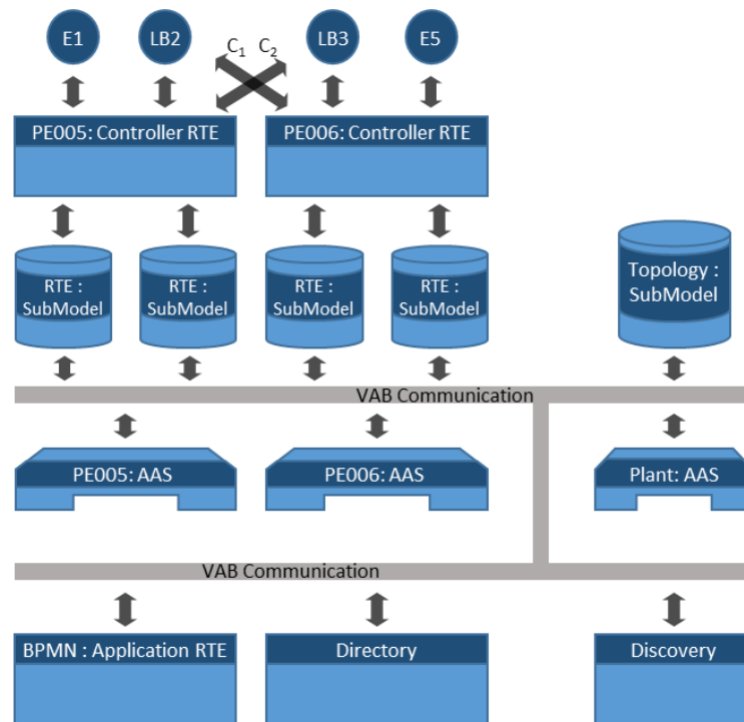
Der Vorteil dieses Ansatzes ist ein reduzierter Aufwand bei der Realisierung unterschiedlicher Kommunikationstechnologien. Anstatt von einer Übersetzung jedes einzelnen Protokolls wird es dadurch möglich die Verwendung von Primitiven, um diesen Prozess zu schlichten und die Komplexität zu reduzieren eclipsefoundationinc.BaSyxEclipsepedia.

In der Abb. 2.9 wird ein Beispiel-Einsatz von einem VAB dargestellt.

2.3.2 Eclipse BaSyx

Eclipse BaSyx ist eine Open-Source-Referenzimplementierung von *BaSys 4.0*-Konzepten, die gängige Industrie 4.0-Komponenten und ein *Software Development Kit (SDK)* enthält, das die Entwicklung neuer Komponenten unterstützt. *Eclipse BaSyx* ist aus dem *BaSys 4.0* Projekt hervorgegangen, das vom Bundesministerium für Forschung und Bildung gefördert wird. Mit seiner Anwendung lassen sich folgende Punkte realisieren:

- *End-to-End*-Digitalisierung der Produktion;
- *End-to-End*-Konnektivität zwischen Werkstatt und IT;
- *Peer-to-Peer*-Kommunikation zwischen Geräten und der IT;
- digitale Prozessmodelle, Wertströme und Produktverfolgung;
- veränderbare Produktionsprozesse bis hin zur Losgröße-Eins-Produktion;
- Zwillinge für Prozesse, Produkte, Geräte und mehr;

Abbildung 2.9: VAB-Beispiel mit verbundenen Entitäten **eclipsefoundationinc.BaSyxEclipsepedia**

- Big-Data-Analyse von Produktionsprozessen;
- vorkonfigurierte Container für Industrie 4.0-Anwendungen, z.B. *Predictive Maintenance*, Analysen und *Dashboards*;
- automatisierte Rückverfolgung und Dokumentation von Produktionsprozessen **eclipsefoundationinc.BaSyxEclipsepedia**.

Diese *Middleware* kann in der Produktion in mehreren Ebenen eingesetzt werden:

1. Feldebene: Die Feldebene besteht aus Automatisierungsgeräten, Sensoren und Aktoren ohne eine spezifische *BaSys*-konforme Schnittstelle;
2. Geräteebene: Die Geräteebene besteht aus Automatisierungsgeräten, die eine *BaSys 4.0*-konforme Schnittstelle anbieten;
3. Middleware-Ebene: Die Middleware-Ebene besteht aus wiederverwendbaren Industrie 4.0-Komponenten, die notwendige generische und anlagen-unabhängige Fähigkeiten für Industrie 4.0-Produktionslinien implementieren;
4. Anlagenebene: Die Anlagenebene besteht aus High-Level-Anlagenkomponenten, die die Produktion verwalten, optimieren und überwachen **eclipsefoundationinc.BaSyxEclipsepedia**.

Zusammenfassend ermöglicht *BaSys* den Aufbau und Einrichten von unternehmensübergreifenden Lösungen, die in der Lage sind, über die ganze Struktur der Fabrik Daten zuverlässig auszutauschen.

Die Abb. 2.10 zeigt eine vereinfachte Struktur, um der Einsatz von den durch *BaSys* implementierten Komponenten (e.g. *AAS*, *registry*-Komponente und *submodels*) zu veranschaulichen **eclipsefoundationinc.BaSyxEclipsepedia**.

Registry-Komponente Die *BaSys 4.0-registry*-Komponente ermöglicht die Registrierung und Suche von AAS innerhalb definierter Systemgrenzen. Entitäten, die AAS einschließlich ihrer Sub-

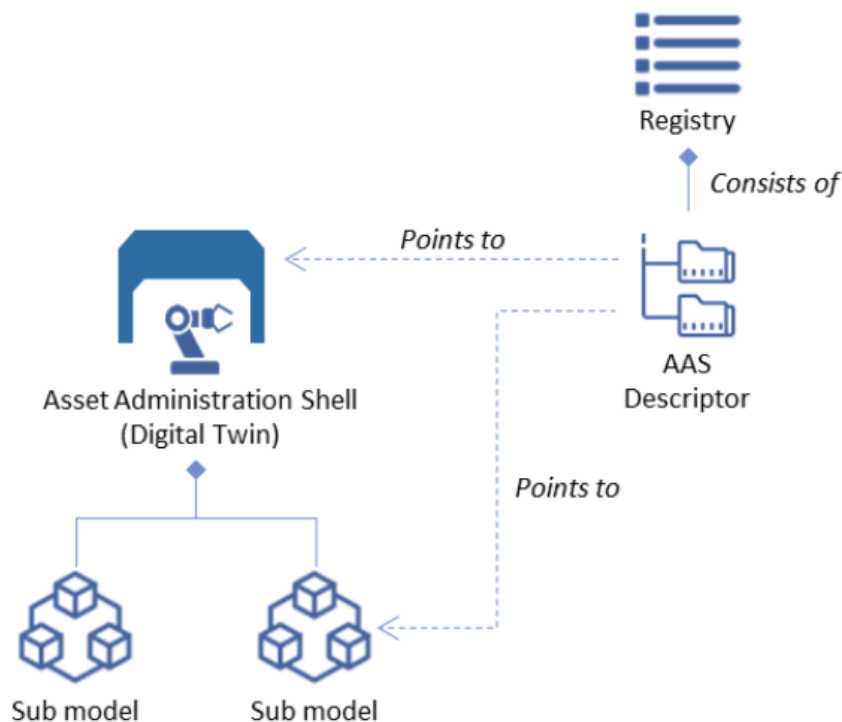


Abbildung 2.10: Diagramm einer Eclipse BaSyx-Systemstruktur
eclipsefoundationinc.BaSyxEclipsepedia

modelle anbieten, können in dieser Komponente registriert werden und damit anderen Teilnehmern ausfindig sein. Die für die Registrierung erforderlichen Informationen sind u.a. eindeutige Identifikatoren für AAS und ihr jeweiliges Asset, Endpunktinformationen zur API der AAS und deren *submodel*-Dienstleister **eclipsefoundationinc.BaSyxEclipsepedia**.

AAS-Server-Komponente Die AAS-Server-Komponente stellt einen AAS-Server zur Verfügung, in dem mehrere AAS und *submodels* gehostet werden können **eclipsefoundationinc.BaSyxEclipsepedia**.

2.4 REST-Schnittstellen

REST ist der am meisten verbreitete Kommunikationsarchitektur zur Vernetzung von Maschinen und wird bei mehreren Anwendungsfällen angewandt, wie z.B. *IoT*-Technologien **reinheimerIndustrieHerausforderungen**. In diesem Zusammenhang wird es zwei Entitäten unterschieden, nämlich dem Server und dem Client. Der letztere initialisiert die Anfrage, während der Server diese bearbeitet und je nach Fall eine Antwort zurückschickt. **balzertLehrbuchSoftwaretechnikEntwurf2011**.

In REST werden Daten über HTTP übertragen und wird es dadurch die Realisierung von Webservices ermöglicht. Ressourcen (jedes Objekt einer Anwendung) repräsentiert eine Ressource, die über eine Adresse (URI, eng.: *Uniform Resource Identifier*) zugreifbar ist **balzertLehrbuchSoftwaretechnikEntwurf2011**. Auf diese Ressource kann eine Anwendung zugreifen, wenn ihr den Identifikator, die zu auszuführende Aktion und das Format der erwartenden Antwort des Servers bekannt sind **balzertLehrbuchSoftwaretechnikEntwurf2011**. Allgemein gilt, dass jede über *HTTP* übertragene Botschaft alle Informationen enthalten darf, die für ihr volle Assimilierung in der Serverseite gebraucht werden, und daher wird die gleiche Anforderung von der REST-Schnittstelle geerbt.

In dieser Architektur werden vier generische HTTP-Methoden zur Verfügung gestellt: *GET*, *PUT*, *POST* und *DELETE*. Diese werden jeweils verwendet, um das Aufrufen, das Hinzufügen, Verarbeiten und das Löschen von Daten im Server durchzuführen **balzertLehrbuchSoftwaretechnikEntwurf2011**.

DRAFT

```

#define S_FUNCTION_NAME timestwo /* Defines and Includes */
#define S_FUNCTION_LEVEL 2

#include "simstruc.h"

static void mdlInitializeSizes(SimStruct *S)
{
    ssSetNumSFcnParams(S, 0);
    if (ssGetNumSFcnParams(S) != ssGetSFcnParamsCount(S)) {
        return; /* Parameter mismatch reported by the Simulink engine */
    }

    if (!ssSetNumInputPorts(S, 1)) return;
    ssSetInputPortWidth(S, 0, DYNAMICALLY_SIZED);
    ssSetInputPortDirectFeedThrough(S, 0, 1);

    if (!ssSetNumOutputPorts(S, 1)) return;
    ssSetOutputPortWidth(S, 0, DYNAMICALLY_SIZED);

    ssSetNumSampleTimes(S, 1);

    /* Take care when specifying exception free code - see sfuntmpl.doc */
    ssSetOptions(S, SS_OPTION_EXCEPTION_FREE_CODE);
}

static void mdlInitializeSampleTimes(SimStruct *S)
{
    ssSetSampleTime(S, 0, INHERITED_SAMPLE_TIME);
    ssSetOffsetTime(S, 0, 0.0);
}

static void mdlOutputs(SimStruct *S, int_T tid)
{
    int_T i;
    InputRealPtrsType uPtrs = ssGetInputPortRealSignalPtrs(S, 0);
    real_T *y = ssGetOutputPortRealSignal(S, 0);
    int_T width = ssGetOutputPortWidth(S, 0);

    for (i=0; i<width; i++) {
        *y++ = 2.0 * (*uPtrs[i]);
    }
}

static void mdlTerminate(SimStruct *S){}

#ifdef MATLAB_MEX_FILE /* Is this file being compiled as a MEX-file? */
#include "simulink.c" /* MEX-file interface mechanism */
#else
#include "cg_sfun.h" /* Code generation registration function */
#endif

```

Kapitel 3

Plannung und Durchführung

Dieser Kapitel ist in zwei Unterkapiteln aufgeteilt, der Entwicklungsprozess und der Entwurf. Im ersteren wird darauf eingegangen, wie die Entwicklung des Projektes abgelaufen ist, mit Emphase auf Anforderungen, Betriebsmodi, funktionelle Architektur und die Softwarestruktur. Im zweiten werden Designaspekten jeder Anforderung beleuchtet, wobei diese technisch beschrieben und mit Auszügen des Quellcodes kontextualisiert werden.

3.1 Entwicklungsprozess

Dieser Teilkapitel erläutert in mehr Detail, wie das System ausgegliedert ist und wie das Projekt entworfen wurde. Im Kapitel 3.1.1 wird das Konzept erläutert, und die für dessen Realisierung notwendigen Anforderungen aufgeführt. Die funktionalen Aspekte des Projektes finden sich im Kapitel 3.1.2 veranschaulicht.

3.1.1 Konzept und Anforderungsdefinition

Die Lösung besteht in einer Bibliothek für Simulink, die mit einem vorhandenen BaSyx-AAS-Server verbindet und das Ablesen und Aktualisieren von dessen Elementen ermöglicht. Dafür wurde auf die verfügbare REST-Schnittstelle vom BaSyx und Matlab S-Funktionen zugegriffen. *S-functions* ergänzen die Simulink-Umgebung, indem personalisierte Blöcke in einer der unterstützenden Programmiersprachen (z.B. C, C++, Fortran und Matlab) entworfen werden können.

Mit der Bibliothek wäre der Benutzer Simulink in der Lage, unkompliziert seinem Modell BaSyx-Blöcken hinzuzufügen und daher mit bestehenden *SubmodelElements* interagieren.

Die Entwicklung der Bibliothek hat vorwiegend zwei Anwendungsfälle berücksichtigt:

1. als Schnittstelle zum Testen von einer Hardware/Software über Simulink im Abbild-Modus, wobei die Simulation mit einem am Anfang aufgenommenen Abbild der Parameterwerte arbeitet, indem das Lesen von Parametern einmal am Anfang der Simulation und Setzen einmal am Ende dieser stattfindet;
2. als Schnittstelle zum Testen von einer Hardware/Software über Simulink mit verlangsamer Simulationstaktung. Dies ermöglicht das in Echtzeit Lesen und Setzen von Parametern des digitalen Zwillings im Laufe der Simulation. Die Abtast-, bzw. Aktualisierungsrate können im Block eingestellt werden.

Aus den Anwendungsfällen lassen sich zwei Betriebsmodi entnehmen: Abbild-Modus für den ersten Anwendungsfall und Betriebsmodus 2: Echtzeitmodus für den zweiten.

Tabelle 3.1: Anforderungstabelle

ID	Anforderung
A10	das System soll den Wert aus einem <i>SubmodelElement</i> ablesen können
A20	das System soll den Wert aus einem <i>SubmodelElement</i> aktualisieren können
A30	das System soll aus einem Registry Server Daten vom einen AAS abrufen können
A40	das System soll Fehler in Simulink-konformen Fehlermeldungen anzeigen können
A50	das System soll Parallelität angemessen implementieren, indem die Verbindung mit mehreren <i>SubmodelElements</i> in einem gleichen Modell möglich ist.
A60	das System soll beide Betriebsmodi (Abbild- und Echtzeitmodus) realisieren.
A70	das System soll eine Bedieneroberfläche im Simulink haben, mit der sich Parameter wie Betriebsmodus, ID von <i>AAS</i> , <i>Submodel</i> , und <i>SubmodelElement</i> einstellen lassen.

Für die ordentliche Realisierung der obenerwähnten Anwendungsfälle setzt sich voraus, dass das System die sämtlichen in der Tabelle 3.1 enthaltenen Anforderungen erfüllen darf.

In diesem Entwicklungsschritt wurden keine Anforderungen bezüglich Grenzen für die Abstrakte von beiden Lese- und Schreibverfahren und der Sicherheit der Datenübertragung festgelegt, stattdessen liegt der Schwerpunkt an der Entwicklung einer ausbaubaren Grundlage für die Bibliothek, die eventuell durch feinere Anforderungen permissiv erweitert werden könnte.

3.1.2 Funktionale Spezifikation

Das System lässt sich in 3 Teilsystemen unterteilen: der (1) *BasyxConnector*, der (2) *HttpClient* und der Blöcke (3) *PropertySAConnectorBlock*.

Der *BasyxConnector* dient als Kommunikationsschnittstellen zwischen einem Block und einem Client. Genauer gesagt werden die durch die sämtlichen für eine Client-Anfrage notwendigen Daten generiert und diese an den intern-gespeicherten Client weitergeleitet. Darüber hinaus wird die Logik für die Kommunikation mit einem *Basyx AAS Server*, bzw. *Basyx-Registry-Server*, in dieser Komponente integriert. Für den Umfang dieses Projektes wurde nur eine Art von *Client* entworfen, bzw. den *HttpClient*, dessen Aufgabe aus sowohl der Abgabe von HTTP-Anfrage wie *GET*, *PUT*, *POST* und *DELETE*, als auch dem Empfangen der Antworten vom Server besteht.

Die *Open-Source* Bibliotheken *libcurl* und *jsoncpp* haben im Hintergrund eine wichtige Rolle bei der Umsetzung der Anforderungen gespielt. Die erstere wird effektiv als Implementierungsgrundlage für den *HttpClient* angewendet, während die letztere zur Dekodierung von HTTP-Antworten verwendet wird. Darüber hinaus wurde eine zusätzliche Komponente zur Bearbeitung von URLs entwickelt. Diese wird durchaus im Projekt verwendet und verfügt über mehrere Methoden, URL automatisch in seinen Bauteilen zu zerlegen und dessen API-Pfad zu bearbeiten.

Alle Blöcke nehmen die Funktionalitäten eines *CommonBlock* im Anspruch, durch den sich die Behandlung von Anfragen an die *Simulink-Engine* implementieren lässt. Derzeit ist eine einzige Implementierung dieser Klasse vorhanden, nämlich der *PropertySAConnectorBlock*, welche in der Lage ist, Leseanfrage und Aktualisierungsanfrage an einen eingegebenen Submodel Element mittels des *BasyxConnector* zu tätigen.

Ein normaler Kommunikationsfluss besteht darin, dass der Block *PropertySAConnectorBlock* (3) in Simulink ausgeführt wird, wodurch nun Methoden vom *BasyxConnector* aufgerufen werden. Dieser setzt im Anschluss den *HttpClient* ein, um effektiv die Http-Anfrage auszusenden. Die Antwort des Servers wird im Anschluss vom *BasyxConnector* abgewickelt und dem ursprünglichen Block entweder den abgerufenen Wert oder eine Fehlermeldung weitergegeben.

Mögliche Events sind das Start-, Takt- und Schluss-Event. Für das Start-Event werden beispielsweise alle durch die Simulink-Oberfläche eingegebenen Parameter im Block internen Speicher abgespeichert. Das Takt-Event wird zu jedem Simulink-Zeitschritt aufgerufen, wobei die aktualisierten Eingabewerte

an dem Block geschickt werden und wiederum eine aktualisierte Ausgabe vom Simulink erwartet wird. Im Abschluss-Event sollen sämtliche initialisierten Objekte im Arbeitsspeicher bereinigt werden.

In der Abb. 3.1 lassen sich die Funktionsbausteine der oben erwähnten Komponenten veranschaulichen.

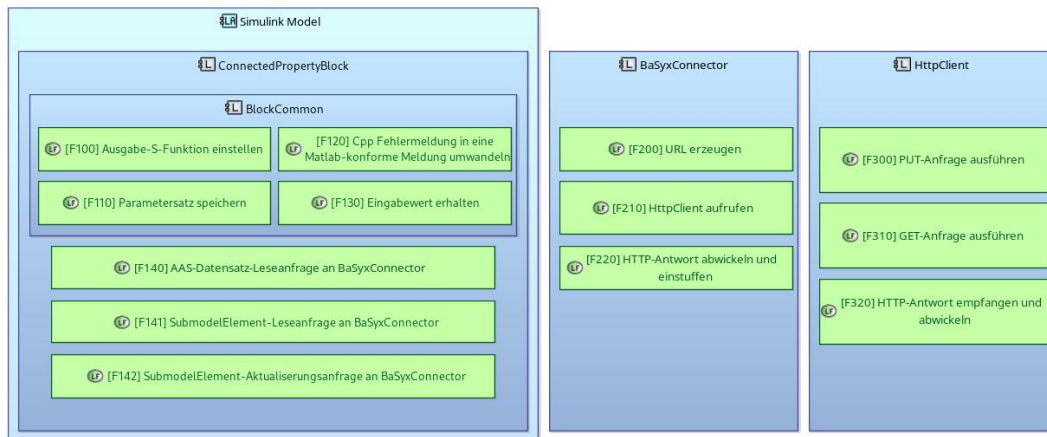


Abbildung 3.1: Funktionsbausteine jeder Teilkomponente

3.1.3 Systemarchitektur

Logische Architektur für das Erfüllen vom A10, A20 und A40

Im Diagramm 3.2 werden alle Kommunikationsflüsse dargestellt, die über eine Verbindung mit dem AAS-Server verfügen. Alle grüne Elemente sind Funktionen der jeweiligen funktionellen Komponente. Außerdem entspricht jede Linienfarbe einem Kommunikationsfluss und daher einer realisierten Fähigkeit.

Die wichtigen Kommunikationsflüsse werden im nächsten Kapitel näher erläutert.

Klassendiagramm

Im Diagramm 3.3 lässt sich ein Überblick der Architektur der Bibliothek erhalten. Zusätzlich zu den internen Komponenten (*BaSyxConnector*, *httpClient*) und den externen Bibliotheken (*jsoncpp*, *curlcpp*, und *simstruc*) wird die Komponente *UrlHandler* im Anspruch genommen, welche für die Bearbeitung von *strings* und Erstellung von *URL* verantwortlich ist.

Wie schon, wird *curlcpp* für das Tätigen von HTTP-Anfragen und *jsoncpp* für die Dekodierung der HTTP-Antworten verwendet. Außerdem legt *simstruc* die Datenstruktur für Simulink fest und kapselt die auf das Modell, bzw. *S-function*, bezogenen Parameter ein.

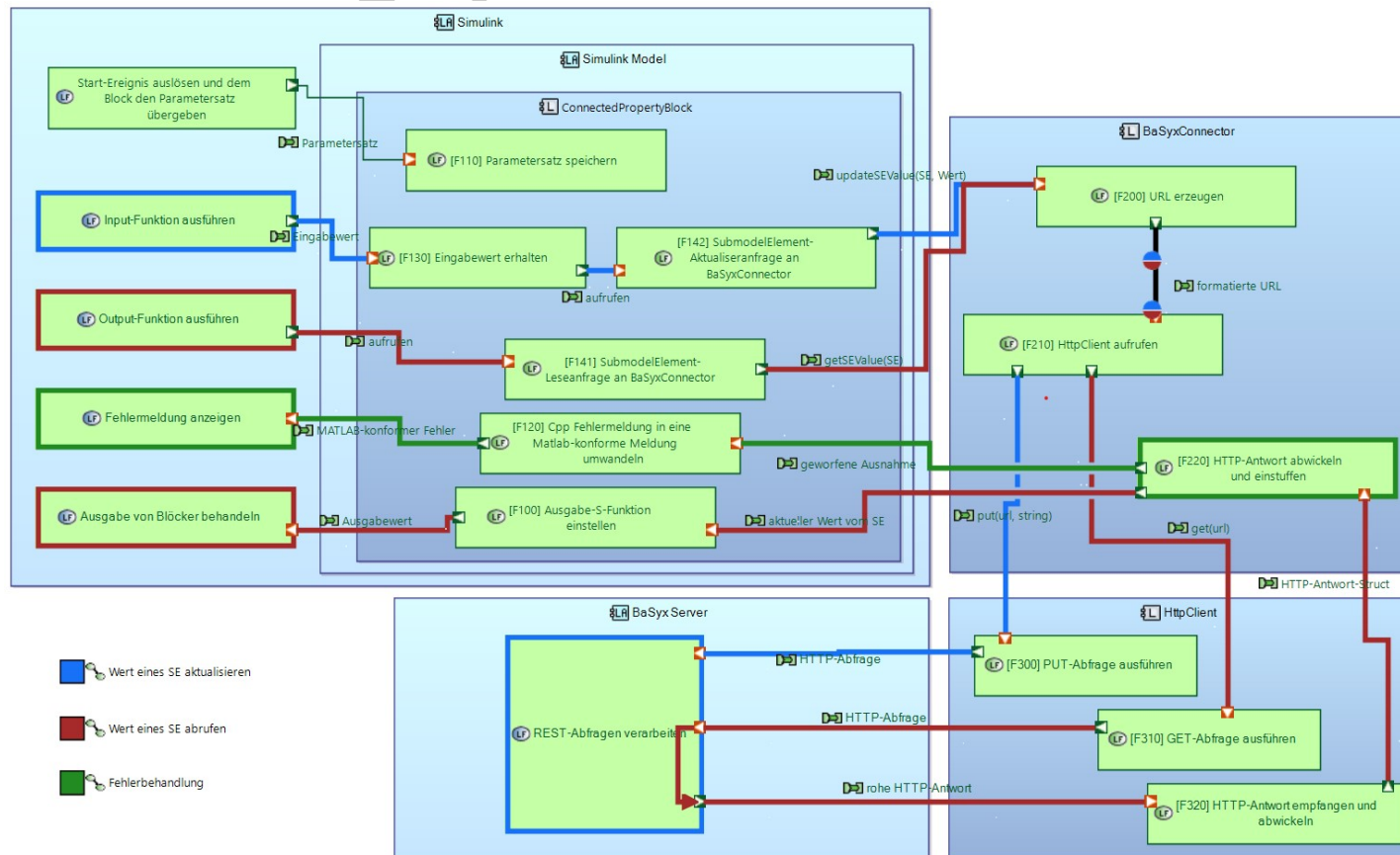


Abbildung 3.2: Darstellung der Kommunikationsflüsse A10, A20 und A40

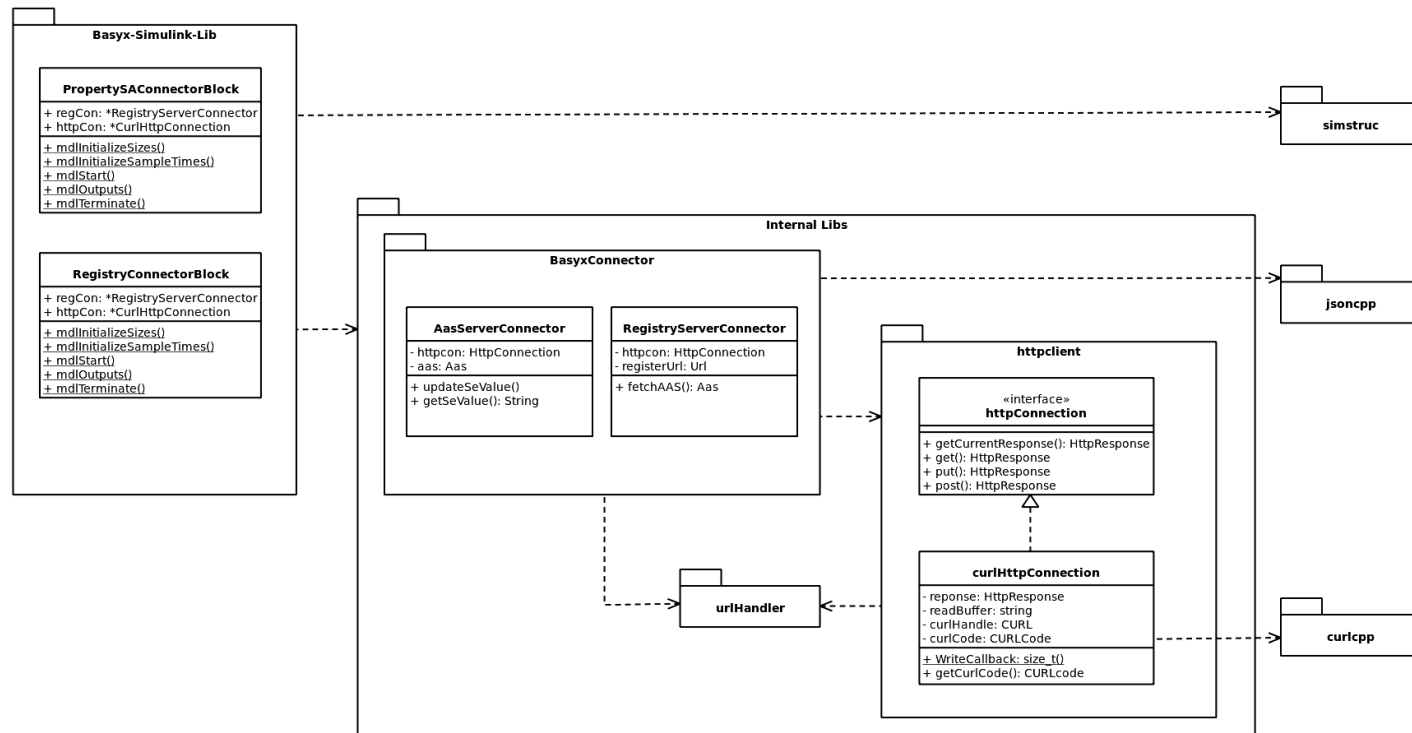


Abbildung 3.3: Klassendiagramm

3.2 Entwurf

Dieser Teilkapitel fängt mit einer Erläuterung davon an, welche Werkzeuge für die Entwicklung des Systems angewendet worden sind. Anschließend wird es auf die Einkapselung von BaSyx in einer *S-function* und auf jede im System implementierte Fähigkeit eingegangen und jeweils das Konzept und Lösungsansatz kurz erklärt. Zum letzten Thema ist ein Überblick über den Entwurf direkt in der Simulationsumgebung vom Simulink gegeben, mit dem Schwerpunkt auf die Personalisierung der Benutzerschnittstelle eines mit *S-function* erstellten Blockes.

3.2.1 Entwicklungsumgebung

Die Bibliothek wurde in C++ mit Visual Studio Code als integrierte Entwicklungsumgebung entwickelt. Zusätzlich war *CMake* durchaus eingesetzt, um das *Packaging* der sämtlichen Module durchzuführen. Für alle Module wurde testgetriebene Entwicklung eingesetzt, um sicherzustellen, dass alle funktionelle und nicht-funktionelle Anforderungen abgedeckt werden. Darüber hinaus wurden beide *google-test* und *google-mock* für die Realisierung von Testfällen angewendet.

Für die Ausführung der Testszenarien sind eine Kombination aus Docker und Simulink verwendet worden. Die BaSyx-Architektur wurde unter Verwendung der von BaSyx veröffentlichten *Docker-Containers* für den *Registry*- und *AAS*-Server bereitgestellt. Außerdem wurden die Verwaltungsschalen mittels *AASX Package Explorer* erstellt, eine Open-Source-Software zur grafischen Visualisierung und Bearbeitung von *AAS/Submodel/SubmodelElements*.

3.2.2 Wrappen des BaSyx als S-Funktion

Der *BasyxConnector* wird innerhalb eines Blockes direkt aufgerufen, um Operationen wie einen AAS aus dem Registry-Server abzurufen und den Wert eines *SubmodelElements* anzufragen, bzw. aktualisieren.

Eine Öffnung der Klasse *aasServerConnector* zeigt auf, dass diese im Konstruktor beide ein Objekt des Typs *AAS* und *HttpConnection* erfordert. Zum Aufbau eines AAS-Objekts wird es auf die Datei *components.hpp* verwiesen, wobei eine neue Instanz entweder aus der Klasse *registryServerConnector* über die Methode *fetchAAS(...)* ausgegeben oder anhand der ID und URL-Pfades erstellt werden kann.

Ein *HttpConnection*-Objekt kann über eine der Implementierungen der Schnittstelle *httpConnection.hpp* angelegt werden. Diese dient zu dem Tätigen von HTTP-Anfragen, wodurch eine effektive Entkopplung von deren Ausführung und dem *BasyxConnector* erreicht wird. Momentan wurde nur eine Implementierung (*curlHttpConnection*) über die externe Bibliothek *libcurl* realisiert.

Um eine Änderung der fest codierten API-Pfaden leichter vorzunehmen, wurden alle API-Pfaden in einer einzigen Datei (Code 2) eingefügt, auf die die sämtlichen Methoden der Bibliothek zugreifen können.

Innerhalb der Datei *PropertySAConnectorBlock.cpp* findet sich die Logik, eine Verbindung mit beiden *BaSyx* und *Simulink* herzustellen. Die Klasse *CurlHttpConnection* wird als Zeiger instanziiert, und als Argument zur Erstellung von Objekten des Paketes *BasyxConnector* über die Methoden *mdlStart* und *mdlOutputs* verwendet.

Um globale Variablen innerhalb der *Simstruc*-Umgebung zu realisieren, werden *Work* Vektoren benötigt. Mit diesen lässt sich Zustand innerhalb einer Block-Instanz speichern und dadurch Initialisierungsparameter wie *aasId*, *registryUrl*, *submodelElementId* unter anderen beibehalten werden. Dafür werden hauptsächlich zwei *simstruct*-Methoden eingesetzt, nämlich *ssGetPWork*, *ssGetPWorkValue*. Die erstere dient zur Bearbeitung des *Work* Vektors, während die letztere zum Ablesen eines in diesem gespeicherten Wert.

```

// BasyxConfig.hpp
...
namespace BasyxConfig
{
    using std::string;
    // Registry Server
    static const string REG_API_PATH = "api/v1/registry";

    // AAS Server

    static const string SM_EPP_ = "submodels/"; //+smIdShort
    static const string SE_EPP_ = "/submodel/submodelElements/"; //+seIdShort
    static const string VALUE_EPP_ = "/value";

    static string buildSeEpPath(string smIdShort, string seIdShort)
    {
        string e_smIdShort = escapeStr(smIdShort);
        string e_seIdShort = escapeStr(seIdShort);

        return SM_EPP_+e_smIdShort+SE_EPP_+e_seIdShort+VALUE_EPP_;
    }
};

```

Codeausschnitt 2: Auszug der API-Definitionen in BasyxConfig

Im Codeauszug 3 lassen sich Beispiele für die Anwendung von *Work* Vektors entnehmen, in dem diese zunächst über die Methode *initializePVectors* initialisiert und später bei jedem Lesevorgang verwendet werden, mit dem Ziel die *AAS*, *Submodel* und *SubmodelElement* Objekten abzurufen.

Die Methode *mdlStart* wird am Moment der Instantiierung des Blockes aufgerufen. Somit werden Objekten wie *AAS*, *Submodel* und *SubmodelElement* angelegt und im internen Zustand über einen *work* Vektor gespeichert. In jedem Simulationsschritt wird die Ausgabe über *mdlOutputs* berechnet. An dem Moment unterscheidet *PropertySAConnectorBlock*, ob eine Verbindung am Eingang, am Ausgang oder an beiden Ports liegt. Je nach Fall wird an den *BasyxConnector* eine Aktualisierungs-, bzw. Leseanfrage übersendet. (s. Code 4)

3.2.3 A10 Ablesen des Wertes aus einem *SubmodelElement*

Für das Ableseverfahren wird es vorausgesetzt, dass eine Verbindung an dem Blockausgang vorliegt. Die Methode *mdlOutputs* wird ausgeführt, und infolgedessen die Methode *getSeValue* des *BasyxConnector* aufgerufen wird. Dieser erzeugt wiederum eine formatierte URL für den *HttpClient*. Anschließend stellt der *HttpClient* eine GET-Anfrage an dem *BaSyx-Server*, und somit dessen Antwort vom *HttpClient* in einem *Struct* mit expliziten Parametern für die rohe Antwort und den *Status-Code* umgewandelt wird (s. Code 5)

Dieses *Struct* wird im *HttpConnection* verarbeitet und im Fall eines *Status-Code* gleich 0 ist der Wert des *SubmodelElement* zurückgegeben. Letztendlich setzt der Block den Wert seines Ausganges zu dem erhaltenen Wert. Das Sequenzdiagramm für den erfolgreichen ausgeführten Prozess lässt sich in der Abb. 3.4 finden.

3.2.4 A20 Aktualisierung des Wertes aus einem *SubmodelElement*

Für das Aktualisierungsverfahren wird vorausgesetzt, dass eine Verbindung an dem Blockeingang vorliegt. Wie bei dem davor beschriebenen Prozess wird die Methode *mdlOutputs* ausgeführt, wobei

```

// PropertySACConnectorBlock.cpp
...
std::unique_ptr<CurlHttpConnection> httpCon;
...
static void initializePVectors(SimStruct *S)
{
    ...
    Aas* aas;
    if(aasUrlStr == ""){
        Url urlStr = Url(regUrlStr);
        RegistryServerConnector regCon = RegistryServerConnector(std::move(urlStr),
→ *httpCon);
        aas = new Aas(regCon.fetchAAS(aasId));
    }else{
        Url urlStr= Url(aasUrlStr);
        aas = new Aas(urlStr, aasId);
    }

    Submodel* sm = new Submodel(*aas,smIdShort);
    SubmodelElement* se = new SubmodelElement(*sm,seIdShort);

    void **PWork = ssGetPWork(S);
    PWork[P_AAS] = aas;
    PWork[P_SM] = sm;
    PWork[P_SE] = se;
}
...
static double readSE(SimStruct *S){
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    return std::stod(aasCon.getSeValue(*se));
}
...

```

Codeausschnitt 3: Auszug der PropertySACConnectorBlock.cpp Datei

die Methode *updateSeValue* des *BasyxConnector* aufgerufen wird. Dieser erzeugt wiederum eine formatierte URL an den *HttpClient*. Anschließend stellt der *HttpClient* eine PUT-Anfrage an dem *BaSyx-Server*. Die Antwort wird letztendlich vom *BasyxConnector* verarbeitet und falls ein Fehler aufgetreten ist, wird eine Ausnahme ausgelöst und der Prozess im 3.2.6 beschrieben wird gestartet. (s. Code 6)

In der Abb. 3.5 findet sich das Sequenzdiagramm von dem oben detaillierten Prozess.

3.2.5 A30 Abrufen von AAS-Daten aus einem Registry Server

Im Gegensatz zu dem Ableseverfahren des Wertes eines *SubmodelElement* wird die Logik für das Abrufen eines AAS-Datensatzes in der Methode *mdlStart* (über die Methode *initializePVectors*) ausgeführt und anschließend wird die formatierte URL über die Methode *fetchAAS* vom *BasyxConnector* erzeugt. Der URL wird am *HttpClient* überreicht, wobei aus der Komponente eine GET-Anfrage an dem *BaSyx-Server* gestellt wird. Die davon erhaltene Antwort ist am Ende vom *BasyxConnector* abgewickelt, und ein Objekt des Typs *AAS* wird am Block zurückgegeben.

```

// PropertySAConnectorBlock.cpp
...
static void mdlOutputs(SimStruct *S, int_T tid)
{
    const double* u = (const double*)ssGetInputPortRealSignal(S, 0);
    double        *y = ssGetOutputPortRealSignal(S,0);
    try{
        // Case snapshot mode
        if(getNumPar(S,PAR_BM)==0){
            y[0] = getLocalSeValue(S);
        }// case real-time mode
        else{
            if (ssGetInputPortConnected(S,0)) updateSE(S,u[0]);
            if (ssGetOutputPortConnected(S,0)) y[0] = readSE(S);
        }
    }catch (const std::exception& e){
        ssSetErrorStatus(S,e.what());
        return;
    }
}
...

```

Codeausschnitt 4: Auszug der PropertySAConnectorBlock.cpp Datei

Der Prozess ist in der Abb. 3.6 veranschaulicht und einen Auszug der Implementierung kann im Codeauszug 7 gefunden werden.

3.2.6 A40 Anzeigen von Simulink-konformen Fehlermeldungen

Wäre ein Status-Code nicht gleich 200 erhalten, löst der *BasyxConnector* eine Ausnahme mit Details des empfangenen Paketes aus, die weiterhin an den entsprechenden Block zugeleitet wird. Für das Wrappen der Ausnahme wird eine Simulink-konforme Methode angewandt, durch die Fehlermeldungen direkt im Simulink-Konsole angezeigt werden können. Die Abb. 3.7 stellt ein Beispiel einer Fehlermeldung, bei der der Server nicht in der Lage war, mit dem Registry-Server eine Verbindung herzustellen.

Diese ist ein wesentlicher Aspekt der Anwendung, was unnötige Abstürze im Programm vermeidet und den Nutzern ermöglicht, Tippfehler schnell zu erkennen und die Verbindung mit dem BaSyx-Server zu überprüfen.

3.2.7 A50 Anwendung von mehreren Blöcken in demselben Simulink-Modell

Aus Sicht Matlab/Simulink ist ein mit *S-function* entwickelter Block eine dynamische geteilte Bibliothek. Eine globale Variable ist deshalb mit allen anderen Blöcken gleiches Typs geteilt, was ein Problem für die Beibehaltung der Daten unter Simulationsschritten darstellt. Wie bereits erwähnt, können zur Umgehung dieses Problems und um den Blöcken einen internen Zustand zu verleihen, *work*-Vektoren verwendet werden.

Das Abbild 3.8 zeigt ein Modell mit mehrfach eingefügten Instanzen des Blockes *PropertySAConnectorBlock*. Jede Instanz funktioniert unabhängig voneinander, trotz unterschiedlichen Abstrakte und auf *SubmodelElement* bezogenen Parametern.

```

// PropertySACConnectorBlock.cpp
...
static double readSE(SimStruct *S){
{
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    return std::stod(aasCon.getSeValue(*se));
}
...

// aasServerConnector.cpp
...
string AasServerConnector::getSeValue(SubmodelElement &se)
{
    ...
    HttpResponse resp = httpCon_.get(aas_.aasUrl);

    if(resp.httpCode != 200 )
        throw std::runtime_error("Unable to get SubmodelElement value at
↪ '"+aas_.aasUrl.getStr()+"'");

    se.updateValue(resp.raw);
    ...
}
...

// httpConnection.hpp
...
struct HttpResponse{
    HttpResponse() : raw(""), httpCode(0) {}
    HttpResponse(string raw, long code) : raw(raw), httpCode(code) {};

    string raw = "";
    long httpCode = 0;
};
...
}

```

Codeausschnitt 5: Auszug der PropertySACConnectorBlock.cpp Datei

3.2.8 A60 Betriebsmodus wechseln

Der Betriebsmodus ist als Parameter im Simulink eingegeben, und anschließend in den *mdlStart*, *mdlOutputs* und *mdlTerminate* Methoden in "PropertySABlockConnector.cpp" geprüft und implementiert. Beim Einschalten vom Abbild-Modus wird die Daten des *Submodelement* nur beim Aufruf vom *mdlStart* und *mdlTerminate* abgerufen, während dies im Echtzeitmodus bei jedem Aufruf von *mdlOutput* geschieht.

3.2.9 A70 Personalisierung der Blockbenutzeroberfläche

Wie im Kapitel 2.2.7 kurz erläutert, bietet Simulink mehrere Möglichkeiten der Benutzeroberfläche eines Blockes zu personalisieren.

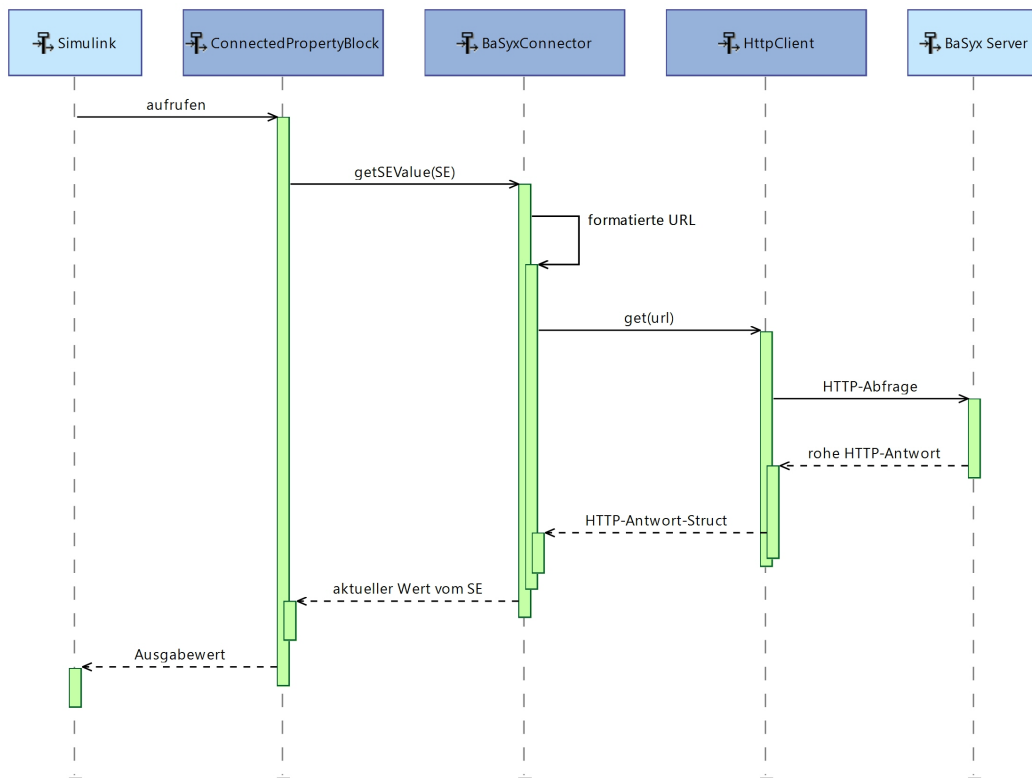


Abbildung 3.4: Sequenzdiagramm für A10

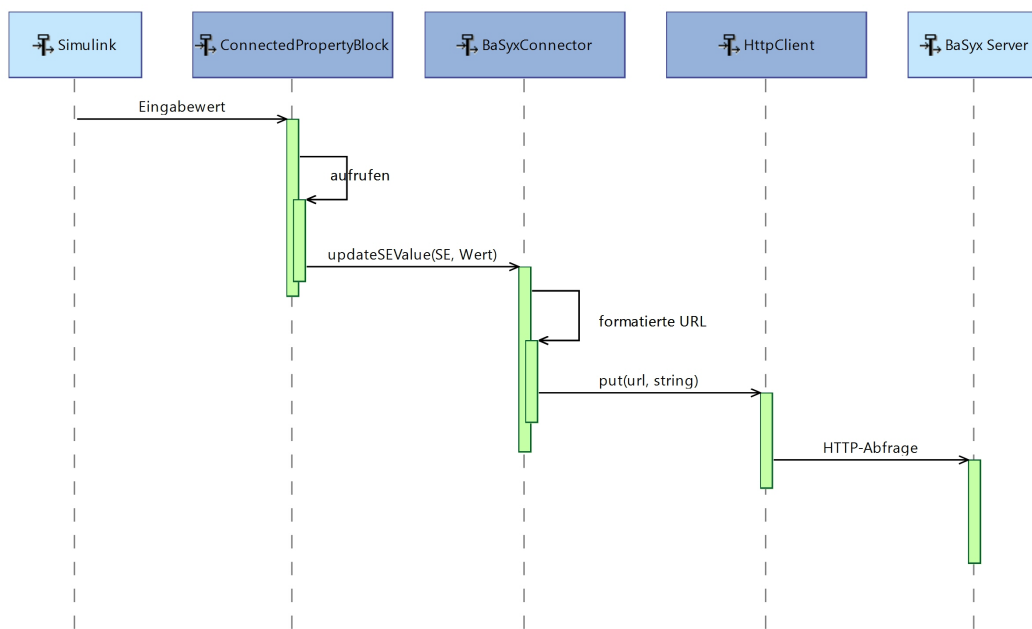


Abbildung 3.5: Sequenzdiagramm für A20

Bei beiden Blöcken war es eine kritische Anforderung, dass alle einzustellenden Parameter gut erkennbar waren, um schnell von mehrfachen eingefügten Blöcken zu unterscheiden.

```

// PropertySAConnectorBlock.cpp
...
static void updateSE(SimStruct *S, double inputValue){
    Aas* aas = (Aas*) ssGetPWorkValue(S,P_AAS);
    SubmodelElement* se =(SubmodelElement*) ssGetPWorkValue(S,P_SE);
    AasServerConnector aasCon = AasServerConnector(*aas, *httpCon);
    aasCon.updateSeValue(*se,std::to_string(inputValue));
}
...

// aasServerConnector.cpp
...
void AasServerConnector::updateSeValue(SubmodelElement &se, const string
→ &value)
{
    ...
    if(httpCon_.put(aas_.aasUrl,value).httpCode == 200 )
        se.updateValue(value);
    else
        throw std::runtime_error("Unable to update SubmodelElement value at
→ '"+aas_.aasUrl.getStr()+"'");
}
...
}

```

Codeausschnitt 6: Auszug der PropertySAConnectorBlock.cpp Datei

```

// PropertySAConnectorBlock.cpp
...
static Aas createAas(string aasUrlStr, string regUrlStr, string aasId){
    // Create AAS based on whether user has provided an AasUrlStr or not
    if(aasUrlStr == ""){
        Url urlStr = Url(regUrlStr);
        RegistryServerConnector regCon = RegistryServerConnector(std::move(urlStr),
→ *httpCon);
        return Aas(regCon.fetchAAS(aasId));
    }else{
        Url urlStr= Url(aasUrlStr);
        return Aas(urlStr, aasId);
    }
}
...

```

Codeausschnitt 7: Auszug der PropertySAConnectorBlock.cpp Datei

Das Block-Symbol wurde personalisiert, um schnell einen Überblick der eingestellten Parameter zu bekommen. Dazu sind Informationen wie die Abtastrate und das verbundene *SubmodelElement* und dessen *Submodel*, bzw. *AAS* immer sichtbar, wie im Bild 3.9 dargestellt ist. Das ermöglicht die schnelle Erkennung und Unterscheidung von mehrfach eingefügten Blöcken in einem Modell.

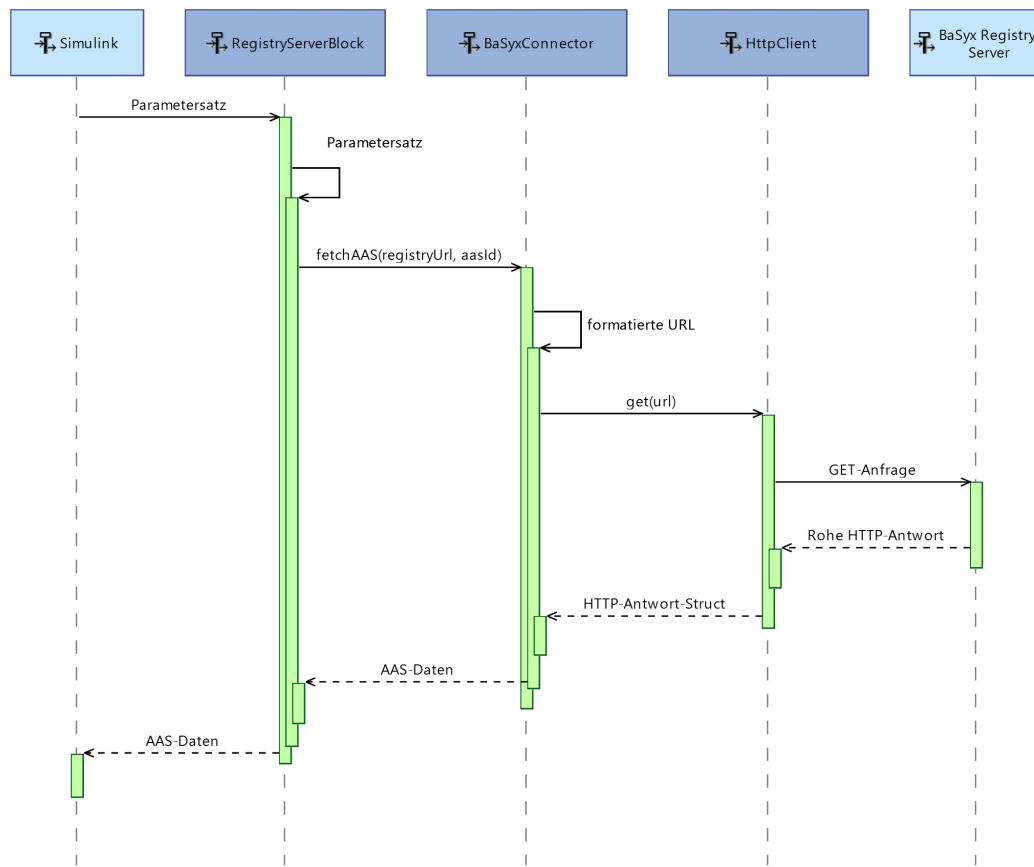


Abbildung 3.6: Sequenzdiagramm für A30

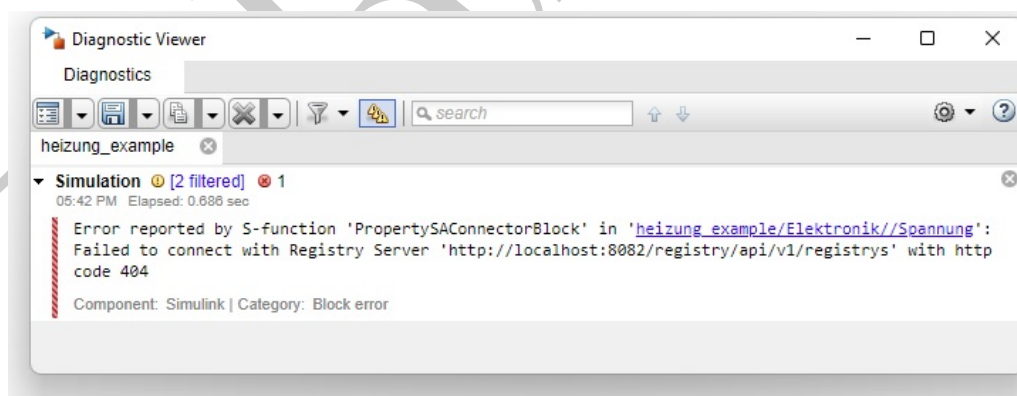


Abbildung 3.7: Beispiel einer Fehlermeldung

Nach Tippen auf einen Block wird dem Benutzer ein Dialogfenster mit Textfelder für alle einzustellende Parameter angezeigt. Das wurde erst durch die Erstellung einer Maske für die *S-function*, eine im Simulink eingebettete Funktionalität. Die Abb. 3.10 zeigt ein Beispiel einer Instanz des Blockes *PropertySAConnectorBlock*.

Der Benutzer kann über das Fenster den gewünschten Betriebsmodus auswählen, wobei bei Wählen des Echtzeitmodus das Feld zur Einstellung der Abtastrate sichtbar wird. Es steht jedoch zur

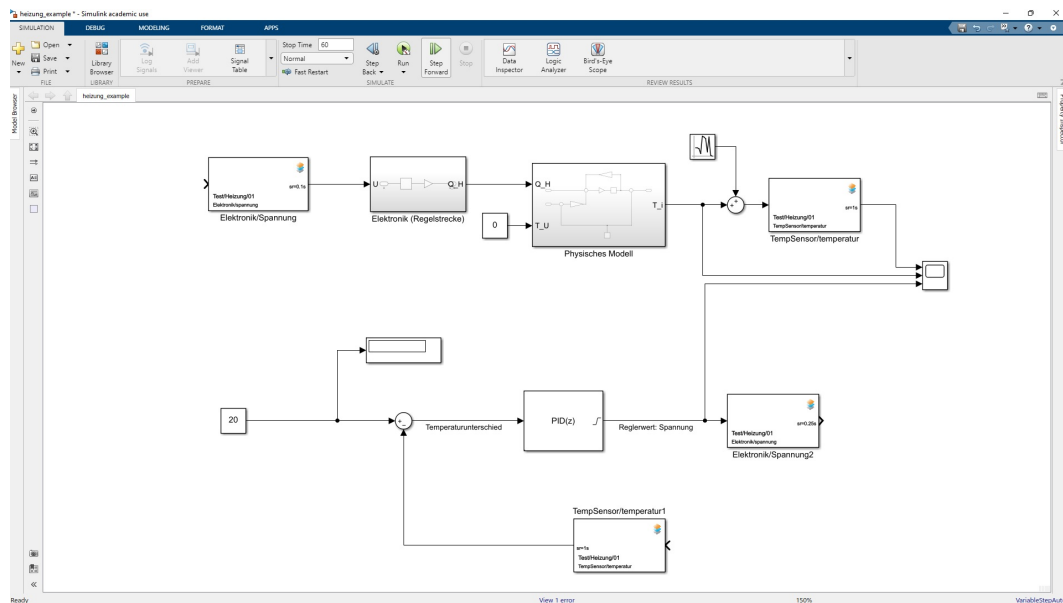


Abbildung 3.8: Simulink-Modell mit mehreren Blöcken

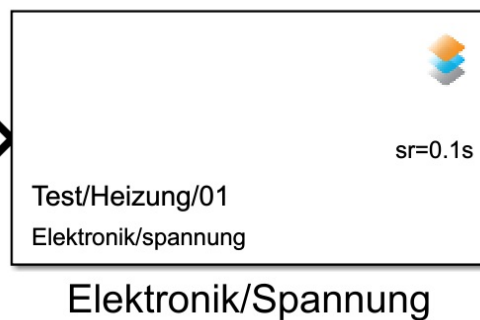


Abbildung 3.9: Instanz eines PropertySACconnector-Block

Verfügung, die Option auszuwählen, ob es die *AAS* über den *RegistryServer* oder direkt über seine Adresse abgerufen werden soll.

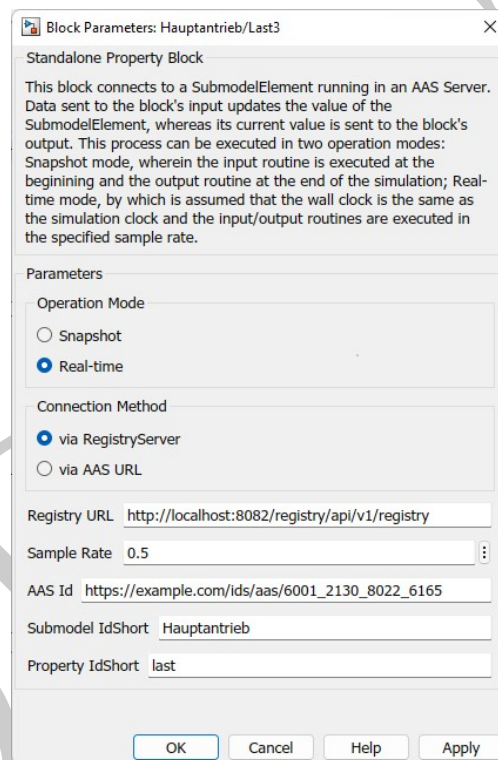


Abbildung 3.10: Dialogfenster des PropertySACconnector-Block

DRAFT

Kapitel 4

Validierung

In diesem Kapitel liegt der Schwerpunkt an der Vorstellung von dem Beispielszenario, genutzt, um die richtige Umsetzung der sämtlichen Anforderungen zu veranschaulichen. Bei dem ersten Teilkapitel wird dessen Aufbau besprochen, während bei dem zweiten und letzten ein Überblick bezüglich der Erwartungen und der aus dem Beispiel ausgegebenen Ergebnisse gewonnen wird.

4.1 Beispielszenario

Für das Testen des Systems wurde ein Beispielszenario entwickelt, bei dem die Fähigkeiten der in der Arbeit entwickelten Bibliothek beobachtet werden können. Dafür wurde ein vereinfachtes Beispiel vom *Condition-Monitoring* realisiert, bei dem die Funktionsweise eines Temperatursensors und des Hauptantriebssystems einer Pressmaschine im Simulink nachgebildet und deren digitale Zwillinge in einem *BaSyx*-Server gespeichert werden. Die Temperaturdaten werden letztendlich in einem in Simulink realisierten Dashboard dargestellt.

Dies bezieht sich auf einen häufigen Anwendungsfall in der Industrie, in dem der Zustand einer Maschine überwacht werden soll, um möglich Ausfälle und Fehler frühzeitig zu erkennen. Im ausgewählten Szenario wirken die Daten aus dem Temperatursensor als Metrik für die Entscheidungsfindung, ob sich die Stanze im normalen Betriebszustand befindet, wobei eine Überschreitung eines voreingestellten Schwellenwerts im einfachsten Fall zu einer Warnung auf dem Dashboard führt.

4.1.1 Setup der *BaSyx*-Infrastruktur mit *Docker*

Beide für das Beispiel verwendeten Registry- und AAS-Server nehmen die verfügbaren *docker containers* von *BaSyx* im Anspruch. Bei Nutzung von *docker-compose* wird eine schnelle Bereitstellung von beiden notwendigen *containers* möglich. Im Codeauszug 8 wird der Aufbau der im Beispiel eingesetzten Konfigurationsdatei für *docker-compose*. Bei Nutzung *Docker-Containers* wird vermieden, dass erst eine komplexere Anwendung in der Form eines *Java*-, bzw. *C*-App erstellt werden muss.

Beim Anfahren lädt der AAS-*container* die im Abb. 4.1 dargestellte AAS. Dieses enthält zwei *Submodels* für vereinfachte Abbildung von beiden Hauptantrieb und Temperatursensor. Diese weisen wiederum je ein *SubmodelElement* auf, die entsprechend zur Darstellung der aktuellen Last am Antriebe und des aktuellen erfassten Temperaturdatenpunktes dienen.

4.1.2 Aufbau

Das Simulink-Modell (Abb. 4.2) besteht aus 4 Teilsystemen:

```
# docker/docker-compose.yml
version: '2.1'
services:
  registry:
    image: eclipsebasyx/aas-registry:1.1.0
    container_name: registry
    volumes:
      - ./conf/reg-config:/usr/share/config:z
    ports:
      - 8082:4000

  aas:
    image: eclipsebasyx/aas-server:1.1.0
    container_name: aas
    volumes:
      - ./conf/aas-config:/usr/share/config:z
    ports:
      - 4001:4001
```

Codeausschnitt 8: Auszug aus der Datei docker/docker-compose.yml

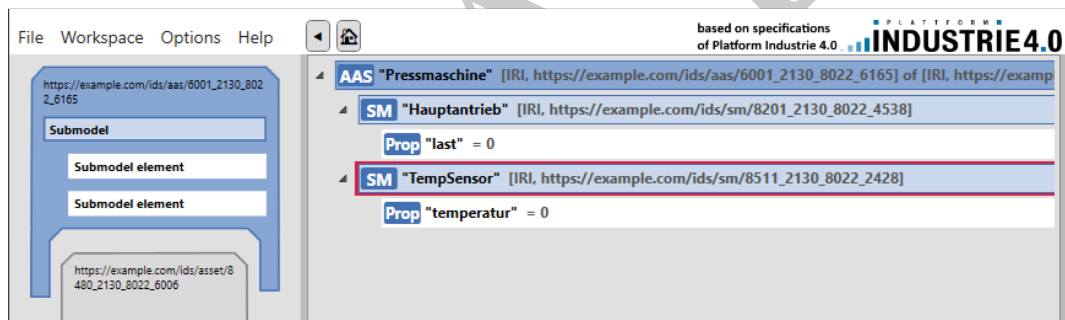


Abbildung 4.1: Bildschirmaufnahme der Verwaltungsschale auf AASX Package Explorer

1. Last: aktualisiert den Wert vom *Hauptantrieb/last SubmodelElement* mit dem Wert einer Schiebekomponente.
2. Hauptantrieb: bildet die Funktionsweise des Hauptantriebes nach und gibt den Augenblickswert der Temperatur am Kontakt zwischen Antriebe und Sensor.
3. Temperatursensor: bildet die Funktionsweise des Temperatursensors und schickt den abgetasteten Wert an *TempSensor/temperatur SubmodelElement*.
4. Dashboard: erhält und verarbeitet die Daten aus dem *TempSensor/temperatur SubmodelElement* und ermöglicht *Monitoring* über einen Graf

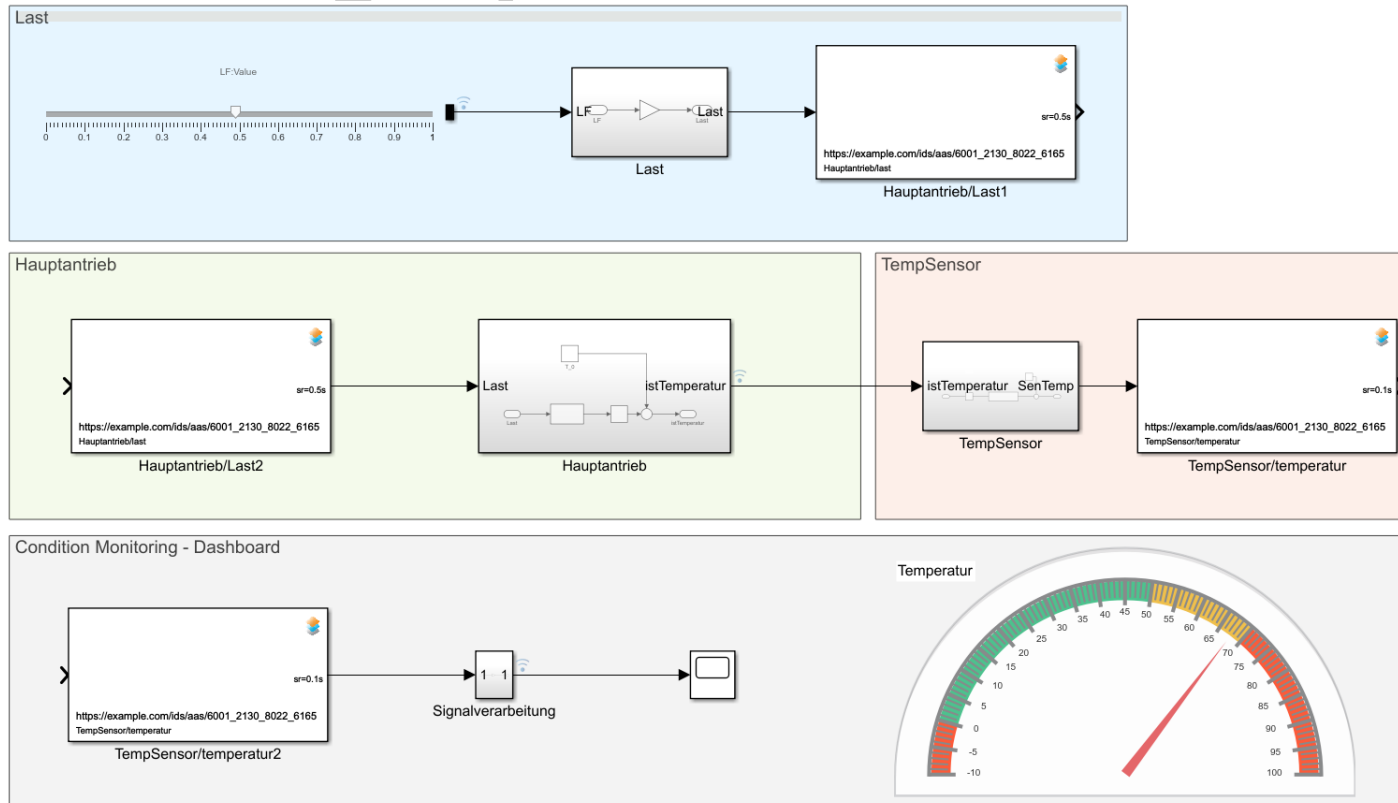


Abbildung 4.2: Simulink-Modell des Beispielszenarios

Beim Einstellen des Schiebers im Last-Teilsystem ändert sich die Temperatur vom Hauptantriebe. Dieser Effekt wird sich auf das Temperatursensor-Teilsystem fortgepflanzt, bei dem die Temperaturdaten erfassen und an den *TempSensor/temperatur SubmodelElement* übertragen werden. Anschließend werden die Daten im Dashboard-Teilsystem über einen zusätzlichen *PropertySAConnectorBlock* angefragt, in einem späteren Block verarbeitet und in einem Graf dargestellt. Der Graf ist in 3 Bereichen aufgeteilt: in grünen Bereich wird die Temperaturen vom normalen Betrieb dargestellt, während in gelben und roten sind entsprechend der Warnzustand und der Notzustand erkennbar.

4.2 Ausführung und Prüfen der Anforderungen

Im Laufe des Tests hat es sich herausgestellt, dass das System in der Lage war, alle Anforderungen angemessen zu realisieren. Das Beispiel wies vielfache Instanzen des Blockes *PropertySAConnectorBlock* auf, und beide Lese- und Aktualisierungsanfrage erfordert hat. Darüber hinaus wurden alle für seine Funktionsweise relevanten Parameter direkt im Simulink eingestellt.

In der Abb. 4.3 werden die Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle bei variiertem Lastfaktor angezeigt. Bei Erreichen vom 70 Grad Celsius, eine willkürlich definierte Grenze, geht die Maschine in einen Notzustand über. Der Sensor ist im Simulink-Modell nicht direkt mit dem Dashboard verbunden, auf diese Weise, läuft die Interaktion nur über seinen entsprechenden digitalen Zwilling durch.

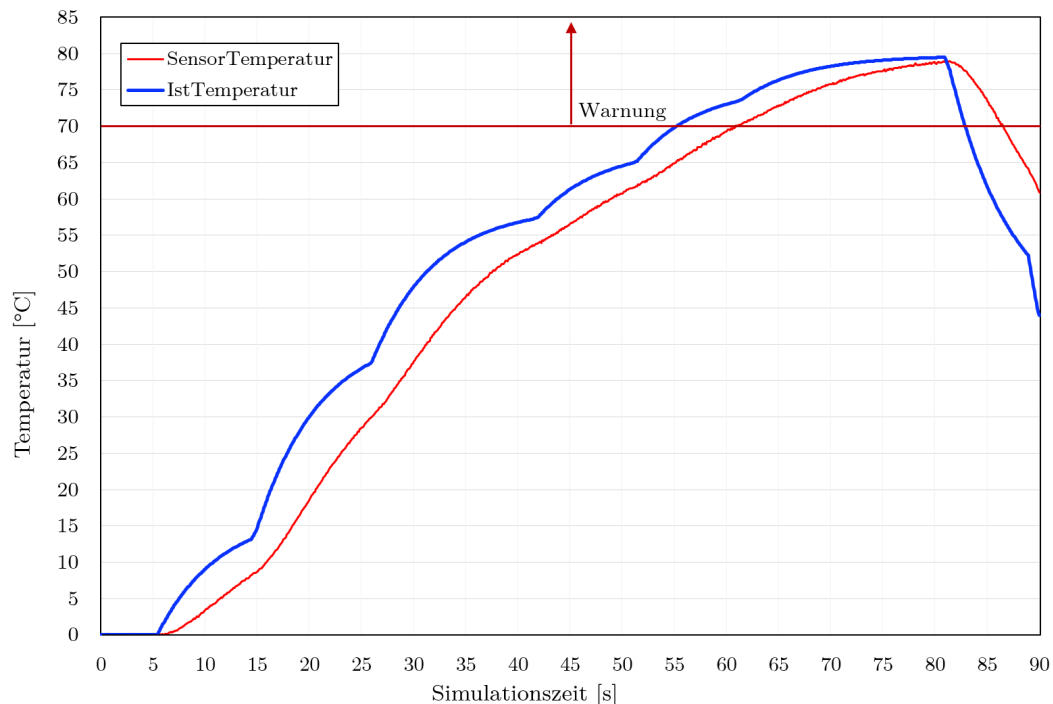


Abbildung 4.3: Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle

Die Simulation wurde im Echtzeitmodus betrieben, bei dem die Simulationszeit der Wanduhrzeit entsprach. Dieser Modus wurde vorgezogen, weil damit sich die Zusammenwirkung BaSyx und Simulink besser vorstellen lässt. Der Abbild-Modus ist prinzipiell sinnvoll, wenn die Simulation asynchron bezüglich der Hardware-, bzw. Software-in-the-loop abläuft.

Es lohnt sich erwähnen, dass soweit Anforderungen bezüglich der höchsten möglichen Abtastrate nicht betrachtet worden sind. Jedoch wurden im Beispiel mehrere Blöcke mit Abtastraten bis 1KHz erfolgreich verwendet.

DRAFT

DRAFT

Kapitel 5

Zusammenfassung und Ausblick

In diesem Kapitel wird eine Zusammenfassung sowohl der wichtigsten geleisteten Punkte der Arbeit, als auch im Ausblick-Abschnitt der in Laufe dieser nicht betrachteten Funktionalitäten. Außerdem werden Beispiele von Erweiterungen aufgeführt, die zu einem reiferen System beitragen könnten.

5.1 Fazit

Diese Arbeit hat seinen Schwerpunkt auf den Konzeptnachweis, dass Simulink-Modelle schnell und einfach an bestehenden BaSyx-Architekturen angeschlossen werden können. In diesem Sinne sind zwei Anwendungsfälle ermöglicht worden: einer für die in Echtzeit Verbindung mit BaSyx-Komponenten, und anderer, Abbild-Modus, für die asynchrone Bearbeitung von Datensätzen aus diesen.

Das ganze System wurde modular entworfen, damit sich künftige Erweiterungen der Fähigkeiten leichter implementieren lassen. Neue Blöcke und Anwendungsfälle können bzw. auf vorhandene Methoden zugreifen, um dadurch über eine getestete Abstraktion der für C++ vorhandene Matlab-Logik verfügen.

Außerdem wurde ein Testszenario für die Validierung des Erfüllens aller festgelegten Anforderungen konzipiert. Dieses Beispiel zeigt weiterhin, wie eine herkömmliche Anwendung des Maschinenbaus und der Verfahrenstechnik mittels Simulink und BaSyx bereitgestellt werden kann.

Die Arbeit zeigt schließlich, dass mit Anwendung der entwickelten Bibliothek *BaSyx* und *Simulink* unkompliziert miteinander verbunden werden können. Sein Einsatz eignet sich besonders gut für das Testen Komponenten in früheren Phasen des Produktlebenszyklus, Demonstratoren aufzubauen und unterschiedliche Varianten eines Konzepts in den Echtbetrieb zu bringen.

5.2 Ausblick

Da das Projekt sich prinzipiell mit dem strukturellen Testen der Anbindung BaSyx und Simulink auseinandergesetzt hat, sollen Fähigkeiten bezüglich der Sicherheit und Ausführungsdauer in späteren Iterationen berücksichtigt werden.

Der einzige in der Arbeit entworfene Block kann verständlich mit Anforderungen anderer Anwendungen nicht völlig nachkommen. Andere Blöcke können somit erstellt werden, um beispielsweise besser Text-Werte zu behandeln und der Kommunikationsfluss weiter zu vereinfachen.

Darüber hinaus wäre es sinnvoll, die Bibliothek in einer anspruchsvolleren Anwendung einzusetzen. Solche Anwendung könnte besser die Vorteile einer Digitalisierungsstruktur mit diesen Werkzeugen hervorheben und genauer deren Grenzen zu erkennen.

DRAFT

Abbildungsverzeichnis

2.1	Blockschaltbild eines Regelungssystems, in dem der Regler im HIL getestet ist ellisControlSystemDesign2012	4
2.2	Struktur eines Simulink-Blockes themathworksinc.WhatSFunctionMATLAB	5
2.3	Struktur eines Simulationssdurchlaufes in Simulink, wobei k als Zähler für den <i>Major</i> -Zeitschritt dient	7
2.4	Ablaufreihenfolge, wenn das Modell initialisiert ist SimulinkEngineInteractiona	8
2.5	Simulationsschleife einer mit C MEX erstellten S-function SimulinkEngineInteractiona	9
2.6	Dialog zur Einstellung der Blockbeutzeroberfläche MaskingFundamentalsMATLAB	10
2.7	Beispiel einer BaSys-Architektur eclipsefoundationinc.BaSyxEclipsepedia	11
2.8	Abbild einer AAS eclipsefoundationinc.BaSyxEclipsepedia	12
2.9	VAB-Beispiel mit verbundenen Entitäten eclipsefoundationinc.BaSyxEclipsepedia	13
2.10	Diagramm einer Eclipse BaSys-Systemstruktur eclipsefoundationinc.BaSyxEclipsepedia	14
3.1	Funktionsbausteine jeder Teilkomponente	19
3.2	Darstellung der Kommunikationsflüsse A10, A20 und A40	20
3.3	Klassendiagramm	21
3.4	Sequenzdiagramm für A10	27
3.5	Sequenzdiagramm für A20	27
3.6	Sequenzdiagramm für A30	29
3.7	Beispiel einer Fehlermeldung	29
3.8	Simulink-Modell mit mehreren Blöcken	30
3.9	Instanz eines PropertySACconnector-Block	30
3.10	Dialogfenster des PropertySACconnector-Block	31
4.1	Bildschirmaufnahme der Verwaltungsschale auf AASX Package Explorer	34
4.2	Simulink-Modell des Beispielszenarios	35
4.3	Verläufe von dem Augenblickswert und dem vom Sensor abgetasteten Wert der Temperatur an der Kontaktstelle	36

DRAFT

Tabellenverzeichnis

3.1 Anforderungstabelle	18
-----------------------------------	----

DRAFT

DRAFT

Titel der Arbeit:

Ermöglichung von Hardware- und Software-in-the-Loop durch
die Integration von BaSyx und MATLAB/Simulink

Art der Arbeit und Datum:

Projektarbeit, August 2022

Betreuung:

Dipl.-Ing. Paaranan Sivasothy
Prof. Dr.-Ing. Jörg Seewig

Student:

Name: Mateus Molina
E-mail: molina@rhrk.uni-kl.de
Matrikelnr.: 420495
Semester: 3

Erklärung betreffend Plagiaten:

Ich erkläre mit meiner Unterschrift die vorliegende Arbeit selbständig verfasst und die im betroffenen Fachgebiet üblichen Zitiervorschriften eingehalten zu haben.

Kaiserslautern, 20. 9. 2022: _____